Type-based Enforcement of Infinitary Trace Properties for Java

Serdar Erbatur
University of Texas at Dallas, USA
serdar.erbatur@utdallas.edu

Ulrich Schöpp
fortiss GmbH, Munich, Germany
schoepp@fortiss.org

Chuangjie Xu
fortiss GmbH, Munich, Germany
xu@fortiss.org

Abstract

The region-based type and effect system for Java by Erbatur, Hofmann and Zălinescu can be used to ensure adherence to programming guidelines by tracking event traces of terminating programs. We refine their system and then extend it to capture also *infinite* traces produced by *non-terminating* programs, based on the ideas of Hofmann and Chen. Our type and effect system for Featherweight Java can express properties of both finite and infinite traces and can compute information about the possible infinite traces of Featherweight Java programs. Specifically, the set of infinite traces of a method is constructed as the greatest fixed point of the operator which calculates the possible traces of method bodies. Our type inference algorithm is realized by working with the *finitary abstraction* of the system based on Büchi automata.

Keywords program analysis, Featherweight Java, effect types, region types, abstract interpretation, Büchi automata

1 Introduction

Developing high quality software is a difficult task that requires systematic approaches to avoiding programming errors. Building on previous experience, software development processes often establish programming guidelines or policies that are designed to rule out errors that are likely to be made. Security guidelines may require one to sanitize all inputs from environment before further processing, they may require that all password changes must be recorded in a log, *etc.* While such guidelines typically capture fairly simple program properties, they help programmers to avoid making typical errors. It would be useful to enforce such guidelines statically and continuously during software development.

At scale, the full automatic verification of Java code is out of scope of current methods. However, due to the relative simplicity of typical guidelines, it is already useful to verify properties of event traces. Let us illustrate the idea using an example. Consider a server that accepts queries for various actions on private data from a user and executes them if the user is authorized to do so. At the end, it logs that sensitive data was accessed.

```
void serve() {
  while(hasQuery()) {
    String query = nextQuery();
    boolean authorized = verifyAuthorization(); // emits authcheck;
    if (authorized) {
      readSensitiveData(); // emits access;
    }
  }
  LogAccess(); // emits log;
}
```

Imagine that functions of interest emit events when they are executed. We can then think of guidelines as specifying which traces of such events are allowed. In the example, we have indicated the emitted events as comments. An example event trace of `serve` would be *authcheck access authcheck log*. It may happen when a user first inputs an authorized request, then an unauthorized one, and then closes the connection. Guidelines, such as "each access to sensitive data occurs after authorization" or "each access to sensitive data is logged", would then be formalized as "*access* must be preceded by *authcheck*" and "each *access* is eventually followed by *log*" respectively.

The example is chosen to illustrate that we want to verify such properties of traces not just of terminating runs, but also of non-terminating ones. The example has intended non-terminating behavior, which repeats the `while`-loop indefinitely. The corresponding traces are infinite repetitions of either *authcheck* or *authcheck access*. This means we want to verify that the infinite words in $\{authcheck, authcheck\ access\}^\omega$ are allowed by the guideline. In addition to such safety properties ("nothing bad happens"), one would also like to verify liveness properties ("something good will happen eventually"). An example for a liveness property would be that any access must be logged, *i.e.* that *access* must be followed by *log*. The above example code would not have this property, since there are infinite traces where no access is logged.

There are a number of options of using automatic verification techniques to verify programming guidelines that have been formalized as sets of allowed traces. The most obvious option would be to use software model checkers. This approach should certainly be expressive enough to handle most guidelines, but it may be limited by scalability problems. One would like to balance expressiveness and scalability. An alternative is to investigate type systems and abstract interpretation. The idea is to develop type systems that are strong enough to express and automatically verify programming guidelines.

In this paper, we define a type system for Featherweight Java (FJ) [12] that allows the automatic analysis of traces of both terminating and non-terminating runs. In previous work, Erbatur et al. [4] have developed a similar type system, which was however limited to finite runs. For infinite runs, [9, 10] have an abstract interpretation approach for the type-based analysis of infinite traces. It is based on Büchi automata, shares their expressiveness and can therefore be used for both safety and liveness properties. However, it was defined for a simple procedural language. While [9, 10] contain a sketch of how to apply their methods in a type system like [4], this development remains at the level of a sketch that needs to be fully developed in further work[1].

Here we develop a type system for FJ that integrates the ideas of both approaches. In contrast to [4], our type system is not a refinement of the FJ type system, but a separate flow type system in the spirit of Microsoft's TypeScript [13] and Facebook's Flow [5]. The resulting type system is simpler, more precise, and more flexible than a refinement of FJ's type system. Our development is based on three main ideas: effect types, regions and abstraction.

Effect Types. Type systems can be extended to statically approximate sets of possible event traces by means of effect types. The idea is to extend the type with effect annotations that give information about the possible traces. This approach was followed in [4], where

[1] Hofmann and Chen [9] do not yet formulate a soundness theorem and the sketched notion of well-typedness is not fully precise (the formulation is unclear w.r.t. the quantification of η). In addition, some choices in the sketched type system seem unnecessarily limiting: For example, any method that calls another method from the same class would be analyzed as having a non-terminating run even when there is no recursion at all.

method types have the form `void serve()` & A, with an *effect annotation* "& A" that gives static information about the possible traces of terminating runs of `serve`. In essence, A represents a set of traces that includes all possible traces of terminating runs. In our example, A could be $\{\mathit{authcheck}, \mathit{authcheck}\,\mathit{access}\}^* \cdot \{\mathit{log}\}$, *i.e.* the set of traces that are a finite repetition of *authcheck* or *authcheck access*, followed by *log*.

Regions. To obtain a useful type system for the analysis of traces in Java, the type system needs to perform some kind of flow analysis. Consider, for example, the Java interface `Runnable` with a single method `void run()`. If we try to annotate this method with an effect annotation for the set of its possible traces, then, without further knowledge, we need to include the traces of all classes that implement `Runnable`. The effect typing of a call like `x.run()` would not be very useful, since it would include the effects of all implementations of `run()`. What is needed is a way to narrow down the possible objects that `x` can point to. This is why [4] integrates both regions and effects in their type system. We also use regions, but diverge from [4] in that we capture them in a separate type system rather than integrating them in the Java type system. Our type system is simpler, more precise and can accommodate richer kinds of regions.

Abstraction. In an implementation of automatic type checking it is of course not possible to store infinite languages like A above directly. Some computable abstraction is needed, a natural choice being finite automata. This choice has been taken for finite words in [4] and for potentially infinite words in [9, 10]. In particular, Hofmann and Chen [9] define an abstraction of Büchi automata for the finite representation of infinite words. Our type system works with the same abstractions. However, the particular choice of abstraction is not important for it. We introduce a notion of Büchi abstraction and establish soundness of the type system for any instance of it. The automata-theoretic constructions of [4, 9] provide an example.

In summary, the contributions of this paper include:

- We introduce a new effect type system that is simpler than those of [4, 9] but allows more precise analysis. It can analyze both terminating and non-terminating programs.
- We give a simple characterization of the infinitary effect analysis as a single greatest fixed point, which allows us to prove its soundness simply by induction on the approximation of the greatest fixed point.
- We realize the type inference by working with the finitary abstraction of the type system following [9]. To have a generic formulation, we introduce a notion of Büchi abstraction that abstracts from the automata-theoretic constructions of [9].

Moreover, we have a prototype implementation of type inference based on Soot [15] that has verified the examples in this paper. Our framework takes a Java bytecode program and a programming guideline as inputs. In particular, the guideline is represented as a Büchi automaton and includes a configuration specifying the default effects of intrinsic methods. For instance, the `verifyAuthorization` method in the `serve` example would be specified to have effect $\{\mathit{authcheck}\}$. From the standard Java typing provided by the Java bytecode, our framework infers the effect information of the program and then verifies if it is accepted by the guideline automaton.

2 Events and Traces

Let Σ be a finite alphabet. We use Σ^* to denote the set of finite words over Σ, and $\Sigma^{\leq\omega}$ the set of finite and infinite words. We write uv for the concatenation of the finite word u with

the word v which can finite or infinite. We now extend concatenation to and recall some other operations on languages: Let $U, U' \subseteq \Sigma^*$ and $V \subseteq \Sigma^{\leq\omega}$. We write

- $U \cdot U' = \{uu' \mid u \in U, u' \in U'\} \subseteq \Sigma^*$,
- $U \cdot V = \{uv \mid u \in U, v \in V\} \subseteq \Sigma^{\leq\omega}$,
- $U^* = \{u_1 u_2 \dots u_n \mid n \in \mathbb{N} \text{ and } u_i \in U \text{ for all } i \in \{1, \dots, n\}\} \subseteq \Sigma^*$,
- $U^+ = U^* \setminus \{\varepsilon\} \subseteq \Sigma^*$, where ε is the empty word, and
- $U^\omega = \{u_1 u_2 \dots u_i \dots \mid u_i \in U \text{ for all } i \in \mathbb{N}\} \subseteq \Sigma^{\leq\omega}$.

We often write a^* rather than $\{a\}^*$ and similarly a^+, a^ω. Note that U^* is the least fixed point of $\lambda X.\{\varepsilon\} \cup U \cdot X$, and U^ω the greatest fixed point of $\lambda X.L \cup (U \setminus \{\varepsilon\}) \cdot X$, where $L = U^*$ if $\varepsilon \in U$ and $L = \emptyset$ otherwise.

For the sake of simplicity, we assume programs have special commands issuing *events* from Σ. Then a terminating execution of a program will generate a finite *trace*, that is, a word from Σ^*, whereas a non-terminating execution will generate a finite or infinite trace, that is, a word from $\Sigma^{\leq\omega}$. Our objective is to capture all possible traces of a given program via a type and effect system, so that we can verify whether they are allowed by a given *guideline*, that is, a language of acceptable traces.

3 Featherweight Java with Updates, Casts and Exception handling

We work with Featherweight Java (FJ) [12] extended with updates, casts and exceptions, and add a primitive operation $\texttt{emit}(a)$ that generates the event a from a finite alphabet Σ. The syntax of the language uses four kinds of names.

variables: $x, y \in \mathit{Var}$, classes: $C, D \in \mathit{Cls}$, fields: $f \in \mathit{Fld}$, methods: $m \in \mathit{Mtd}$.

Program expressions are defined as follows:

$$\begin{aligned} \mathit{Expr} \ni e ::= {} & x \mid \texttt{let}\ x = e_1\ \texttt{in}\ e_2 \mid \texttt{if}\ x = y\ \texttt{then}\ e_1\ \texttt{else}\ e_2 \mid \texttt{null} \mid \texttt{new}^\ell\ C \\ & \mid (C)\, e \mid x^C.f \mid x^C.f := y \mid x^C.m(\bar{y}) \mid \texttt{emit}(a) \\ & \mid \texttt{throw}\ e \mid \texttt{try}\ e_1\ \texttt{catch}(C\, x)\ e_2 \end{aligned}$$

The expression $\texttt{new}^\ell\ C$ is annotated with a label ℓ. We use labels only to distinguish different occurrences of `new` in a program; since our type system will track where objects were created. In a few expressions we have added type annotations and write x^C for a variable of type C. When they are not needed, we shall write annotations with invisible ink.

We assume the distinguished elements $\texttt{this} \in \mathit{Var}$ and $\texttt{Object} \in \mathit{Cls}$. We further assume a formal element $\texttt{NullType} \in \mathit{Cls}$ that plays the role of the type of `null` from the Java language specification [7, §4]. It may not be used in programs, *i.e.* we require $C \neq \texttt{NullType}$ in the expressions $\texttt{new}^\ell\ C$ and $(C)\, e$. When x is not a free variable of e_2, we may write $e_1; e_2$ rather than $\texttt{let}\ x = e_1\ \texttt{in}\ e_2$.

A Featherweight Java program $(\prec, \mathit{fields}, \mathit{methods}, \mathit{mtable})$ consists of

- a subtyping relation $\prec \in \mathcal{P}^{\text{fin}}(\mathit{Cls} \times \mathit{Cls})$ with $C \prec D$ meaning that C is an immediate subclass of D,
- a field list $\mathit{fields} : \mathit{Cls} \to \mathcal{P}^{\text{fin}}(\mathit{Fld})$ mapping a class to its fields,
- a method list $\mathit{methods} : \mathit{Cls} \to \mathcal{P}^{\text{fin}}(\mathit{Mtd})$ mapping a class to its methods,
- a method table $\mathit{mtable} : \mathit{Cls} \times \mathit{Mtd} \rightharpoonup \mathit{Var}^* \times \mathit{Expr}$ mapping a method to the pair of its formal parameters and its body.

All components are required to be *well-formed*. We refer the reader to *e.g.* [4, Section 3] for details. Let $\preceq$ be the reflexive and transitive closure of $\prec$. We have $C \preceq \texttt{Object}$ and $\texttt{NullType} \preceq C$ for all $C \in \mathit{Cls}$.

▶ **Example 1.** As a running example, we consider the following Java code, in which ℓ_1, ℓ_2 and ℓ_3 are arbitrary fresh labels.

```
class Node {
  Node next;
  Node last() {
    emit(a);
    if (next == null) {
      return this;
    } else {
      return next.last();
    }
  }
}
```

```
class Test {
  Node linear() {
    Node x = new^ℓ1 Node();
    Node y = new^ℓ2 Node();
    y.next = x;
    return y.last();
  }
  Node cyclic() {
    Node z = new^ℓ3 Node();
    z.next = z;
    return z.last();
  }
}
```

In Featherweight Java, the class `Node` is represented as follows.

$$\begin{aligned} \mathit{fields}(\texttt{Node}) &= \{\texttt{next}\} \\ \mathit{methods}(\texttt{Node}) &= \{\texttt{last}\} \\ \mathit{mtable}(\texttt{Node},\texttt{last}) &= ((), e_{\texttt{last}}) \end{aligned}$$

$$\begin{aligned} e_{\texttt{last}} := \ & \texttt{let } _ = \texttt{emit}(a) \texttt{ in} \\ & \texttt{let } x = \texttt{this}^{\texttt{Node}}.\texttt{next} \texttt{ in} \\ & \texttt{let } y = \texttt{null} \texttt{ in} \\ & \texttt{if } x = y \texttt{ then this} \\ & \texttt{else let } z = \texttt{this}^{\texttt{Node}}.\texttt{next} \texttt{ in } z^{\texttt{Node}}.\texttt{last}() \end{aligned}$$ ◀

In the standard type system of Featherweight Java, types are simply classes. It is modeled by a class table (F_0, M_0), where the *field typing* $F_0 : \mathit{Cls} \times \mathit{Fld} \rightharpoonup \mathit{Cls}$ assigns to each class C and field $f \in \mathit{fields}(C)$ the class of f, and the *method typing* $M_0 : \mathit{Cls} \times \mathit{Mtd} \times \mathit{Cls}^* \rightharpoonup \mathit{Cls} \times \mathit{Cls}^*$ specifies for each class C, method $m \in \mathit{Mtd}(C)$ and classes of the argument variables the classes of the result value of m and of the exceptions that m may throw. The typings are required to be invariant w.r.t. superclass (see *e.g.* [12]). In the rest of this paper, we consider only well-typed FJ programs.

The operational semantics of Featherweight Java uses a notion of state (s, h) consisting of a *store* $s : \mathit{Var} \rightharpoonup \mathit{Val}$ mapping variables to values and a *heap* $h : \mathit{Loc} \rightharpoonup \mathit{Obj}$ mapping locations to objects. The only kinds of values are object locations or *null*. An object contains a class identifier and a valuation of its fields.

$$\begin{array}{lll} \text{locations:} & l \in \mathit{Loc} & \text{stores:} \quad s \in \mathit{Var} \rightharpoonup \mathit{Val} \\ \text{values:} & v \in \mathit{Val} = \mathit{Loc} \uplus \{\mathit{null}\} & \text{heaps:} \quad h \in \mathit{Loc} \rightharpoonup \mathit{Obj} \\ \text{objects:} & \mathit{Obj} = \mathit{Cls} \times (\mathit{Fld} \rightharpoonup \mathit{Val}) & \end{array}$$

There are three relations:

- $(s, h) \vdash e \Downarrow v, h' \,\&\, w$ means that, in state (s, h), the expression e evaluates to the value v with the heap updated to h', generating the event trace $w \in \Sigma^*$.
- $(s, h) \vdash e \downharpoonright l, h' \,\&\, w$ means that, in state (s, h), the expression e throws an exception that is stored in the location l, with the heap updated to h', generating the event trace $w \in \Sigma^*$.
- $(s, h) \vdash e \Uparrow \,\&\, w$ means that, in state (s, h), the evaluation of e diverges and generates the trace $w \in \Sigma^{\leq \omega}$.

The operational semantics of terminating evaluation and of exception throwing are defined mutually, and the one of nonterminating execution is generated coinductively. The complete list of operational semantics rules is given in Appendix A.

4 Region Types and Effects

We introduce a type system for region and effect analysis for FJ. In contrast to previous work [4, 9], we do not refine FJ type system, but instead build an independent region typing system for FJ. Region type information is complementary to FJ type information and can be captured without repeating the FJ type system. This also makes sense practically: we would not want to modify the Java compiler but develop an additional analysis system.

For the sake of understandability, in this section we present the system with arbitrary languages over Σ as effect annotations. In Section 5 we shall make it algorithmic by moving to a finitary abstraction of languages.

4.1 Region Types

Our type system captures region information. A *region* represents provenance information about a value, such as where in the program an object was created or what its actual class is. The intention is that the set of values is subdivided in regions according to various criteria.

In this paper, we use the following definition of regions:

$$\mathit{Reg} \ni r, s \ ::= \ \texttt{Null} \mid \texttt{CreatedAt}(\ell) \mid \texttt{Unknown}$$

These regions have the following meaning. The region `Null` contains only the value *null*. The region $\texttt{CreatedAt}(\ell)$ contains all references to objects that were created by an expression of the form $\texttt{new}^{\ell}\ C$. This region allows us to track where in the program an object originates. The region `Unknown` is for references to objects of unknown origin, *e.g.* from library code.

It is possible to use richer definition of regions to capture other properties of interest, such as taintedness, or to give regions more structure. For example, one may allow unions or intersections of regions. We plan to do in further work. Here we use a simple representative definition of regions in order to focus on the new type system itself and on infinitary traces.

Regions provide class information about the actual classes of objects. We write $\mathit{Cls}(r)$ for the set of possible classes of an object in region r, which is defined as follows: $\mathit{Cls}(\texttt{Null}) = \emptyset$, $\mathit{Cls}(\texttt{CreatedAt}(\ell)) = \{C \mid \text{program contains } \texttt{new}^{\ell}\ C\}$, and $\mathit{Cls}(\texttt{Unknown}) = \mathit{Cls}$. Note that that $\mathit{Cls}(r)$ is *not* required to be closed under superclass. Indeed, $\mathit{Cls}(\texttt{CreatedAt}(\ell))$ will be just a singleton if the label ℓ was used only once.

Regions also provide information about the identity of objects. Objects in disjoint regions cannot be identical. The region `Null` and all regions of the form $\texttt{CreatedAt}(\ell)$ are all pairwise disjoint. However, `Unknown` overlaps with all other regions. We write $\mathit{disjoint}(r, s)$ to indicate that two regions are disjoint.

4.2 Region and Effect Expressions

Before diving into the details of the type system, we explain the region and effect expressions that are used in the type system. The typing judgment will have the following form:

$$\Gamma \vdash e : T \ \mathsf{throws}\ H \ \mathsf{calls}\ S.$$

The context Γ simply maps program variables to regions. More interesting are T, H and S on the right-hand side. They are expressions that contain region and effect information about the computation of e. In this section we explain the meaning of these expressions.

First, both T and H are expressions of the form $r_1\ \&\ U_1 \mid \cdots \mid r_n\ \&\ U_n$. The former lists the possible options for the terminating computations of e. The option $(r_i\ \&\ U_i) \in T$

means that the result value is in region r_i and the computation trace is in $U_i \subseteq \Sigma^*$. Thus, T expresses that whenever e evaluates to a value, there will be some $(r_i \,\&\, U_i) \in T$ such that the result is in region r_i and the computation trace is in U_i. Similarly, H expresses that whenever e throws an exception, there will be some $(r_i \,\&\, U_i) \in H$ such that (the value of) the exception is in region r_i and the computation trace is in U_i.

For example, the expression T for the following FJ-program

$$\texttt{let}\ x = \texttt{if}\ \mathit{cond}\ \texttt{then}\ (\texttt{emit}(a); \texttt{new}^{\ell_1}\ C)\ \texttt{else}\ (\texttt{new}^{\ell_2}\ D)\ \texttt{in}\ \texttt{emit}(b);\ x$$

can be $\texttt{CreatedAt}(\ell_1)\ \&\ \{ab\} \mid \texttt{CreatedAt}(\ell_2)\ \&\ \{b\}$. It either evaluates to a reference pointing to an object that was created by a `new` with label ℓ_1 or to one that was created by a `new` with label ℓ_2. In the first case, the trace of the computation is ab, and in the latter case it is just b. It may also be given the less precise type $\texttt{Unknown}\ \&\ \{ab, b\}$, which just expresses that it produces some value and either has effect ab or b while doing so.

Second, S is a *call expression* of the form $U_1 \cdot \delta_1 \cup \cdots \cup U_k \cdot \delta_k$. It contains information about the calls that e may make. In the expression, each δ_i is a method signature, which is given by a class name, a method name and region information. We specify signatures precisely below, but the details are not important for a first explanation of S. The expression S gives us information about which method calls e can make and what the traces leading up to such calls can be. The expression states that for any call that e makes, there exists an i such that the event trace leading to the call is in U_i and the call goes to a method with signature δ_i.

Consider, for example, the following FJ-expression e:

$$\texttt{emit}(a);\ x.\texttt{f}();\ \texttt{emit}(b);\ x.\texttt{g}()\ .$$

If we assume that `f` and `g` have no effects on their own, then the call expression S for e would be $\{a\} \cdot \delta_{\texttt{f}} \cup \{ab\} \cdot \delta_{\texttt{g}}$, where $\delta_{\texttt{f}}$ and $\delta_{\texttt{g}}$ are the signatures of `f` and `g` respectively.

The information about possible method calls will be useful to approximate the traces of non-terminating computations. A non-terminating computation in Featherweight Java must consist of an infinite series of method calls. To compute an approximation of all infinite traces of such sequences, it suffices to compute the finite effect traces from one method call to the next and to consider their concatenation. We do this by computing an expression like S for the body of each method (Section 4.3). Then we compute an approximation of all infinite traces from this information (Section 4.4).

To work with the expressions T, H and S, we need some notation. While we use different notations for expressions to emphasize their different meaning, they are in fact instances of the same kind of formal expressions.

▶ **Definition 2** (Formal effect expression). *Given a set K of keys, we define the set $\mathcal{P}(\Sigma^*)\langle K\rangle$ of* formal effect expressions *to be the set of finite partial functions from K to $\mathcal{P}(\Sigma^*)$. We write $\emptyset$ to denote the empty expression. For $T, T' \in \mathcal{P}(\Sigma^*)\langle K\rangle$ and $U \in \mathcal{P}(\Sigma^*)$, we define:*

- *$T \subseteq T'$ if and only if $T(x) \subseteq T(x')$ for all $x \in K$;*
- *$T \cup T' \in \mathcal{P}(\Sigma^*)\langle K\rangle$ by $(T \cup T')(x) = T(x) \cup T'(x)$;*
- *$U \cdot T \in \mathcal{P}(\Sigma^*)\langle K\rangle$ by $(U \cdot T)(x) = U \cdot T(x)$.*

We use the notation $r_1 \,\&\, U_1 \mid \cdots \mid r_n \,\&\, U_n$ to represent the elements of $\mathcal{P}(\Sigma^*)\langle \mathit{Reg}\rangle$. Such an expression denotes the function mapping r_i to U_i for $i = 1, \ldots, n$. For example, we have $(r_1 \,\&\, U_1 \mid r_2 \,\&\, U_2) \cup (r_1 \,\&\, U_1' \mid r_3 \,\&\, U_3) = (r_1 \,\&\, (U_1 \cup U_1') \mid r_2 \,\&\, U_2 \mid r_3 \,\&\, U_3)$ and $U \cdot (r_1 \,\&\, U_1 \mid r_2 \,\&\, U_2) = (r_1 \,\&\, (U \cdot U_1) \mid r_2 \,\&\, (U \cdot U_2))$. Likewise, the call expressions $U_1 \cdot \delta_1 \cup \cdots \cup U_n \cdot \delta_n$ are elements of $\mathcal{P}(\Sigma^*)\langle \mathit{Sig}\rangle$, where Sig is a set of method signatures (to be defined below). When $U_i = \{\varepsilon\}$ we may omit it and simply write δ_i in the expressions.

4.3 Typing Rules

Having explained region and effect expressions, we can now define the type system in detail. As for Featherweight Java, we need a *class table* to record the region and effect expressions of methods and fields. This is needed to formulate typing rules for method call and field access.

We call a tuple $(C, r, m, \bar{s})$ of a class C, a region r, a method $m \in \mathit{methods}(C)$ and a sequence $\bar{s}$ of regions for the arguments of m a *method signature*. We use $\mathit{Sig} \subseteq \mathit{Cls} \times \mathit{Reg} \times \mathit{Mtd} \times \mathit{Reg}^*$ to denote the set of method signatures.

▶ **Definition 3** (Class table). *A* class table (F, M) *consists of*

- *a* field typing $F : \mathit{Cls} \times \mathit{Reg} \times \mathit{Fld} \rightharpoonup \mathcal{P}(\mathit{Reg})$ *that assigns to each class* C*, region* r *and field* $f \in \mathit{methods}(C)$ *a set* $F(C, r, f)$ *of possible regions of the field* f*, and*
- *a* method typing $M : \mathit{Sig} \to \mathcal{P}(\Sigma^*)\langle \mathit{Reg} \rangle \times \mathcal{P}(\Sigma^*)\langle \mathit{Reg} \rangle \times \mathcal{P}(\Sigma^*)\langle \mathit{Sig} \rangle$ *that assigns to each method signature a tuple of expressions for terminating executions, exception throwing and call effects.*

A class table is required to be well-formed *in the following sense:*

- $\texttt{Null} \in F(C, r, f) \subseteq F(C, \texttt{Unknown}, f)$*;*
- $F(C, r, f) = F(D, r, f)$ *whenever* $C \preceq D$ *and* $f \in \mathit{fields}(D)$*; and*
- $M(C, r, m, \bar{s}) \subseteq M(D, r, m, \bar{s})$ *whenever* $C \preceq D$ *and* $m \in \mathit{methods}(D)$*, where the order* $\subseteq$ *on* $\mathcal{P}(\Sigma^*)\langle \mathit{Reg} \rangle \times \mathcal{P}(\Sigma^*)\langle \mathit{Reg} \rangle \times \mathcal{P}(\Sigma^*)\langle \mathit{Sig} \rangle$ *is given componentwise.*

The well-formedness conditions reflect the subtyping properties of FJ. We additionally require all entries of F to contain the region `Null`, because the fields of newly created objects are all initialized to *null* (see the operational semantics in Appendix A). We require F to be invariant w.r.t. also the "largest" region `Unknown`. Note that the tables may contain impossible entries whose class and its subclasses are not in the region. They are not important in the typing inference. We can set them to the "bottom". A typical example is $M(C, \texttt{Null}, m, \bar{s}) = (\emptyset, \emptyset, \emptyset)$.

We denote table entries $F(C, r, f) = R$ and $M(C, r, m, \langle s_1, \ldots, s_n \rangle) = (T, H, S)$ as

```
class C@r
  f : R
  m(s1,...,sn) : T throws H calls S
```

with which we hope to improve readability.

The typing judgments take the form $\Gamma \vdash e : T \text{ throws } H \text{ calls } S$, where $\Gamma : \mathit{Var} \rightharpoonup \mathit{Reg}$ is a typing environment, $e \in \mathit{Expr}$ a term expression, $T, H \in \mathcal{P}(\Sigma^*)\langle \mathit{Reg} \rangle$ formal expressions for terminating executions and exception throwing respectively, and $S \in \mathcal{P}(\Sigma^*)\langle \mathit{Sig} \rangle$ an expression for call effects.

The typing rules are given in Figure 1. Rule WEAK says that any possible region r can be "weakened" to `Unknown`. It allows us to merge the sets of possible traces. For instance, if $e : r \,\&\, U \mid \texttt{Unknown} \,\&\, U'$ then by WEAK we have $e : \texttt{Unknown} \,\&\, (U \cup U')$. Rule PRIM says that $\texttt{emit}(a)$ has the terminating trace a and no exceptions or calls. Rules NULL, NEW and GET have similar effects. In rule LET, e_1 may have multiple regions and effects, so we type e_2 with x in each of these regions r_i to get the effects of e_2 and then take the union of them. In rule CALL for method call, we simply look up the method typing M for the terminating effect and take the method signature as the call expression. Rule TRYCATCH is similar to rule LET, but e_2 is typed only with the regions r_i of e_1 to which the exception class E is related, *i.e.*, E or its subclass is allowed by r_i. Moreover, because all the exceptions of or below class E that e_1 may throw would be caught, we should remove the related exception effects before joining them into the resulting one. For this, we define an operation H/E to remove from H those pairs $(r \,\&\, U)$ such that all the classes allowed by r are below E, *i.e.*, we define $H/E = H/\{(r \,\&\, U) \in H \mid C \preceq E \text{ for all } C \in \mathit{Cls}(r)\} \in \mathcal{P}(\Sigma^*)\langle \mathit{Reg} \rangle$.

$$\textsc{sub}\ \frac{\Gamma \vdash e : T \ \mathsf{throws}\ H \ \mathsf{calls}\ S \quad (T,H,S) \subseteq (T',H'.S')}{\Gamma \vdash e : T' \ \mathsf{throws}\ H' \ \mathsf{calls}\ S'}$$

$$\textsc{weak}\ \frac{\Gamma \vdash e : (r\ \&\ U) \cup T \ \mathsf{throws}\ H \ \mathsf{calls}\ S}{\Gamma \vdash e : (\mathtt{Unknown}\ \&\ U) \cup T \ \mathsf{throws}\ H \ \mathsf{calls}\ S}$$

$$\textsc{prim}\ \frac{}{\Gamma \vdash \mathtt{emit}(a) : \mathtt{Null}\ \&\ \{a\} \ \mathsf{throws}\ \emptyset \ \mathsf{calls}\ \emptyset}$$

$$\textsc{var}\ \frac{}{\Gamma,\, x{:}\, r \vdash x : r\ \&\ \{\varepsilon\} \ \mathsf{throws}\ \emptyset \ \mathsf{calls}\ \emptyset}$$

$$\textsc{if}\ \frac{\Gamma,\, x{:}\, r,\, y{:}\, s \vdash e_1 : T_1 \ \mathsf{throws}\ H_1 \ \mathsf{calls}\ S_1 \qquad \Gamma,\, x{:}\, r,\, y{:}\, s \vdash e_2 : T_2 \ \mathsf{throws}\ H_2 \ \mathsf{calls}\ S_2}{\Gamma,\, x{:}\, r,\, y{:}\, s \vdash \mathtt{if}\ x = y\ \mathtt{then}\ e_1\ \mathtt{else}\ e_2 : T_1 \cup T_2 \ \mathsf{throws}\ H_1 \cup H_2 \ \mathsf{calls}\ S_1 \cup S_2}$$

$$\textsc{else}\ \frac{\mathit{disjoint}(r,s) \qquad \Gamma,\, x{:}\, r,\, y{:}\, s \vdash e_2 : T_2 \ \mathsf{throws}\ H_2 \ \mathsf{calls}\ S_2}{\Gamma,\, x{:}\, r,\, y{:}\, s \vdash \mathtt{if}\ x = y\ \mathtt{then}\ e_1\ \mathtt{else}\ e_2 : T_2 \ \mathsf{throws}\ H_2 \ \mathsf{calls}\ S_2}$$

$$\textsc{let}\ \frac{\begin{array}{c}\Gamma \vdash e_1 : r_1\ \&\ U_1 \mid \ldots \mid r_n\ \&\ U_n \ \mathsf{throws}\ H \ \mathsf{calls}\ S \\ \Gamma,\, x{:}\, r_i \vdash e_2 : T_i \ \mathsf{throws}\ H_i \ \mathsf{calls}\ S_i\end{array}}{\Gamma \vdash \mathtt{let}\ x = e_1\ \mathtt{in}\ e_2 : \bigcup_{i=1}^n U_i \cdot T_i \ \mathsf{throws}\ H \cup \bigcup_{i=1}^n U_i \cdot H_i \ \mathsf{calls}\ S \cup \bigcup_{i=1}^n U_i \cdot S_i}$$

$$\textsc{null}\ \frac{}{\Gamma \vdash \mathtt{null} : \mathtt{Null}\ \&\ \{\varepsilon\} \ \mathsf{throws}\ \emptyset \ \mathsf{calls}\ \emptyset}$$

$$\textsc{new}\ \frac{}{\Gamma \vdash \mathtt{new}^{\ell}\ C : \mathtt{CreatedAt}(\ell)\ \&\ \{\varepsilon\} \ \mathsf{throws}\ \emptyset \ \mathsf{calls}\ \emptyset}$$

$$\textsc{cast}\ \frac{\Gamma \vdash e : T \ \mathsf{throws}\ H \ \mathsf{calls}\ S}{\Gamma \vdash (D)\, e : T \ \mathsf{throws}\ H \ \mathsf{calls}\ S}$$

$$\textsc{call}\ \frac{(T,H,_) = M(C,r,m,\bar{s})}{\Gamma,\, x{:}\, r,\, \bar{y}{:}\, \bar{s} \vdash x^C.m(\bar{y}) : T \ \mathsf{throws}\ H \ \mathsf{calls}\ (C,r,m,\bar{s})}$$

$$\textsc{set}\ \frac{s \in F(C,r,f)}{\Gamma,\, x{:}\, r,\, y{:}\, s \vdash x^C.f := y : s\ \&\ \{\varepsilon\} \ \mathsf{throws}\ \emptyset \ \mathsf{calls}\ \emptyset}$$

$$\textsc{get}\ \frac{T = \bigcup\{s\ \&\ \{\varepsilon\} \mid s \in F(C,r,f)\}}{\Gamma,\, x{:}\, r \vdash x^C.f : T \ \mathsf{throws}\ \emptyset \ \mathsf{calls}\ \emptyset}$$

$$\textsc{throw}\ \frac{\Gamma \vdash e : T \ \mathsf{throws}\ H \ \mathsf{calls}\ S}{\Gamma \vdash \mathtt{throw}\ e : \emptyset \ \mathsf{throws}\ T \cup H \ \mathsf{calls}\ S}$$

$$\textsc{trycatch}\ \frac{\begin{array}{c}\Gamma \vdash e_1 : T \ \mathsf{throws}\ H \ \mathsf{calls}\ S \qquad H = r_1\ \&\ U_1 \mid \ldots \mid r_n\ \&\ U_n \\ \Gamma,\, x{:}\, r_i \vdash e_2 : T_i \ \mathsf{throws}\ H_i \ \mathsf{calls}\ S_i \text{ and } C \preceq E \text{ for some } C \in \mathit{Cls}(r_i)\end{array}}{\begin{array}{l}\Gamma \vdash \mathtt{try}\ e_1\ \mathtt{catch}(E\,x)\ e_2 : T \cup \bigcup_{i=1}^n U_i \cdot T_i \ \mathsf{throws}\ H/E \cup \bigcup_{i=1}^n U_i \cdot H_i \\ \qquad\qquad \mathsf{calls}\ S \cup \bigcup_{i=1}^n U_i \cdot S_i\end{array}}$$

Figure 1 Region-based type and effect system

▶ **Example 4.** Suppose there are two classes $D \prec C$ with two methods f and g. Consider the following class table:

```
class C@CreatedAt(ℓ1)                 class D@CreatedAt(ℓ2)
  f() : Null & {a} throws ∅ calls ...   f() : Null & {b} throws ∅ calls ...
  g() : Null & {aa} throws ∅ calls ...  g() : Null & {bb} throws ∅ calls ...
```

Following the typing rules, the FJ-expression $e \equiv \texttt{if}\ \mathit{cond}\ \texttt{then}\ (\texttt{new}^{\ell_1}\ C)\ \texttt{else}\ (\texttt{new}^{\ell_2}\ D)$ has type $\texttt{CreatedAt}(\ell_1)\ \&\ \{\varepsilon\} \mid \texttt{CreatedAt}(\ell_2)\ \&\ \{\varepsilon\}$. Now consider the following expressions:

$$\texttt{let}\ x = e\ \texttt{in}\ x.f() \qquad \texttt{let}\ x = e\ \texttt{in}\ x.f(); x.f() \qquad \texttt{let}\ x = e\ \texttt{in}\ x.g()$$

The first expression has type $\texttt{Null}\ \&\ \{a, b\}$, the second and third one have type $\texttt{Null}\ \&\ \{aa, bb\}$. One may have expected the traces of the second expression to be $\{a, b\} \cdot \{a, b\}$, that is, $\{aa, ab, ba, bb\}$, because x may be in two different regions in each call. Indeed, this is the case in the system of [4], but *not* in ours. According to the LET rule, $x.f(); x.f()$ is typed twice with the two regions of x, and the results $\{aa\}$ and $\{bb\}$ are joined in the end, resulting in $\{aa, bb\}$.

This seems to be slight defect of [4]. Indeed, the function g could have been defined to have body $\texttt{this}.f(); \texttt{this}.f()$. In this case, the second expression would be the inlining of the third and one would perhaps expect inlining not to lose information. ◀

▶ **Definition 5** (Well-typedness). *A program* $(\prec, \mathit{fields}, \mathit{methods}, \mathit{mtable})$ *is* well-typed *w.r.t. a class table* (F, M) *if for every* $(C, r, m, \bar{s}) \in \mathit{Sig}$ *with* $C \in \mathit{Cls}(r)$ *and* $M(C, r, m, \bar{s}) = (T, H, S)$ *and* $\mathit{mtable}(C, m) = (\bar{x}, e)$*, the judgment* $\texttt{this}: r,\ \bar{x}: \bar{s} \vdash e : T$ throws H calls S *is derivable.*

The program from Example 1 is well-typed, details appear in Example 8 below.

One subtle aspect of well-typedness is the condition $C \in \mathit{Cls}(r)$. This condition allows us to use the region information to statically narrow down the possible targets of calls. This is an improvement over [4], where the information was not used. With the condition, well-typedness makes no requirement on $M(C, r, m, \bar{s})$ when $C \notin \mathit{Cls}(r)$. This is because the table entry is for a method that cannot be executed in region r. The following example illustrates how our type system uses region information to narrow the possible targets of call expressions.

▶ **Example 6.** Consider a program with two simple classes (the return type is `Object` only because we do not have a `void` type):

```
class A {                          class B extends A {
  Object f() {                       Object f() {
    emit(a);                           emit(b);
    return null;                       return null;
  }                                  }
}                                  }
```

Let ℓ_1 and ℓ_2 be labels such that $\mathit{Cls}(\texttt{CreatedAt}(\ell_1)) = \{\texttt{A}\}$ and $\mathit{Cls}(\texttt{CreatedAt}(\ell_2)) = \{\texttt{B}\}$. Suppose the program is well-typed with respect to the following class table:

```
class A@Unknown                    class B@Unknown
  f(): Null & {a, b} throws ∅ calls ∅   f(): Null & {b} throws ∅ calls ∅

class A@CreatedAt(ℓ1)              class B@CreatedAt(ℓ1)
  f(): Null & {a} throws ∅ calls ∅      f(): ∅ throws ∅ calls ∅

class A@CreatedAt(ℓ2)              class B@CreatedAt(ℓ2)
  f(): Null & {b} throws ∅ calls ∅      f(): Null & {b} throws ∅ calls ∅
```

The types in region `Unknown` should be unsurprising. The requirement that the typings be closed under supertyping requires us to include the effect b also in class A. In regions

$\texttt{CreatedAt}(\ell_1)$ and $\texttt{CreatedAt}(\ell_2)$ we can ignore the bodies of `B.f` and `A.f` respectively when constructing the method table. This is because the definition of well-typedness for programs makes a requirement only for entries $(C, r, m, \bar{s})$ with $C \in \mathit{Cls}(r)$. Here we have $\texttt{B} \notin \mathit{Cls}(\texttt{CreatedAt}(\ell_1))$ and $\texttt{A} \notin \mathit{Cls}(\texttt{CreatedAt}(\ell_2))$, so we can ignore the bodies of `B.f` and `A.f` in regions $\texttt{CreatedAt}(\ell_1)$ and $\texttt{CreatedAt}(\ell_2)$ respectively. Moreover, the entry with class `B` at region $\texttt{CreatedAt}(\ell_1)$ is impossible and thus set to be bottom $(\emptyset, \emptyset, \emptyset)$.

With the above class table, consider the following code fragment.

```
A x = new^ℓ2 B();
x.f();
```

The type system allows us to give `x` type $\texttt{CreatedAt}(\ell_2)$. In this case, the method call `x.f()` is analyzed to have effect $\{b\}$, which is exact. We could also give `x` the type `Unknown`, but this choice would lead to an over-approximation of the actual effect. We would get that the method call `x.f()` has effect $\{a, b\}$. However, we would not be allowed to give `x` the type $\texttt{CreatedAt}(\ell_1)$. ◀

4.4 Infinite Traces

We now turn to analyzing possibly infinite traces of non-terminating programs. These cannot be captured directly from an inductive typing derivation. Instead, we use the type system to justify the call expressions (see Definition 5 of well-typed programs) and subject them the following *coinductive* treatment which will lead to the correctness of the non-terminating effects (Theorem 10).

▶ **Definition 7.** *Given a method typing M, we write S_δ to denote the call expressions in $M(\delta)$. We call an assignment $\eta : \mathit{Sig} \to \mathcal{P}(\Sigma^{\leq\omega})$ an* infinitary effect typing for M *if it is the* greatest solution *of the equation system $\{\delta = S_\delta \mid \delta \in \mathit{Sig}\}$.*

In the above definition, we treat method signatures as variables and thus call expressions as $\mathcal{P}(\Sigma^{\leq\omega})$-valued functions. We will prove the correctness of the non-terminating effects by induction on an approximation to the greatest solution η, but begin with an example for the definition.

▶ **Example 8.** A possible class table for the program from Example 1 is shown below. It uses the regions $r_1 = \texttt{CreatedAt}(\ell_1)$, $r_2 = \texttt{CreatedAt}(\ell_2)$ and $r_3 = \texttt{CreatedAt}(\ell_3)$. In the table we use the abbreviation $\delta_r = (\texttt{Node}, r, \texttt{last}, ())$.

```
class Node@Null
  next : {Null}
  last() : ∅ throws ∅ calls ∅

class Node@r1
  next : {Null}
  last() : r1 & {a} throws ∅ calls {a} · δ_Null

class Node@r2
  next : {Null, r1}
  last() : r1 & {aa} | r2 & {a} throws ∅ calls {a} · δ_r1

class Node@r3
  next : {Null, r3}
  last() : r3 & a+ throws ∅ calls {a} · δ_r3

class Test@Unknown
  linear() : r1 & {aa} | r2 & {a} throws ∅ calls δ_r2
  cyclic() : r3 & a+ throws ∅ calls δ_r3
```

The members of class `Node` are given different types depending on the region. In region r_1, the field `next` has region `Null`. This means that `next` can only be *null*, since the location of any object would abstract to some region rather than `Null`. As a result, method `last` will always have a as its trace, thus its terminating effect is $\{a\}$. In region r_2, the field `next` has region $\{\texttt{Null}, r_1\}$. If method `last` returns `this` then it has region r_2 and effect $\{a\}$; otherwise, it returns `next.last()` and thus has region r_1 and effect $\{aa\}$ where the second a is emitted by `last` at r_1. This explains the annotation $r_1 \,\&\, \{aa\} \mid r_2 \,\&\, \{a\}$. In region r_3, we have a circularity, as `next` has also region r_3.

We have the following equation system from the method typing:

$$(\texttt{Test}, \texttt{Unknown}, \texttt{linear}, ()) = \delta_{r_2} \qquad \delta_{\texttt{Null}} = \emptyset \qquad \delta_{r_1} = \{a\} \cdot \delta_{\texttt{Null}}$$
$$(\texttt{Test}, \texttt{Unknown}, \texttt{cyclic}, ()) = \delta_{r_3} \qquad \delta_{r_2} = \{a\} \cdot \delta_{r_1} \qquad \delta_{r_3} = \{a\} \cdot \delta_{r_3}$$

Its greatest solution η maps $\delta_{\texttt{Null}}$, δ_{r_1}, δ_{r_2} and $(\texttt{Test}, \texttt{Unknown}, \texttt{linear}, ())$ to $\emptyset$, and δ_{r_3} and $(\texttt{Test}, \texttt{Unknown}, \texttt{cyclic}, ())$ to a^ω. Therefore, η is suitable as an infinitary effect typing. ◀

4.5 Soundness

We follow [4, Appendix E.2] to interpret the typing judgment using heap typings.

▶ **Definition 9.** *A* heap typing $\Pi : \mathit{Loc} \rightharpoonup \mathit{Reg}$ *assigns to each heap location a region for objects. A heap typing* Π' extends *a heap typing* Π*, written* $\Pi' \sqsupseteq \Pi$*, if* $\mathrm{dom}(\Pi) \subseteq \mathrm{dom}(\Pi')$ *and* $\Pi(l) = \Pi'(l)$ *for all* $l \in \mathrm{dom}(\Pi)$*.*

A typing judgment for values takes the form $\Pi \vdash v : r$*, meaning that the value* v *is in region* r *according to the heap typing* Π*. Here are the typing rules for values:*

$$\frac{}{\Pi \vdash \mathit{null} : \texttt{Null}} \qquad \frac{l \in \mathrm{dom}(\Pi)}{\Pi \vdash l : \Pi(l)} \qquad \frac{\Pi \vdash v : r}{\Pi \vdash v : \texttt{Unknown}}$$

We lift it to stores and environments: $\Pi \vdash s : \Gamma$ *iff* $\Pi \vdash s(x) : \Gamma(x)$ *for all* $x \in \mathrm{dom}(\Gamma)$*.*

Lastly, we say a heap h *is* well-typed *w.r.t. a heap typing* Π *(and implicitly a class table* (F, M)*), written* $\Pi \vdash h$*, if* $l \in \mathrm{dom}(h)$ *and* $\Pi \vdash h(l) : \Pi(l)$ *for all* $l \in \mathrm{dom}(\Pi)$*, where* $\Pi \vdash (C, G) : r$ *iff* $C \in \mathit{Cls}(r)$ *and* $\mathrm{dom}(G) = \mathit{fields}(C)$ *and* $\Pi \vdash G(f) : t$ *for some* $t \in F(C, r, f)$ *and for all* $f \in \mathit{fields}(C)$*.*

Roughly speaking, an object is well-typed if the value of each field is well-typed, and a heap is well-typed if the object at each location of the heap is well-typed.

Now we interpret the typing judgment $\Gamma \vdash e : T$ throws H calls S in the form of a soundness theorem. It states that whenever a well-typed program is executed on a heap that is well-typed w.r.t. a heap typing Π, then the computation trace is captured by the type, and the resulting heap after the execution is well-typed w.r.t. some possibly larger heap typing $\Pi' \sqsupseteq \Pi$ if the execution terminates or throws an exception.

▶ **Theorem 10** (Soundness)**.** *Let* P *be a well-typed program w.r.t. the class table* (F, M)*. Let* η *be an infinitary effect typing for* M*. For any* Γ*,* e*,* T*,* H*,* S*,* Π*,* s*,* h*,* v*,* h' *and* w *with*

$$\Gamma \vdash e : T \ \mathsf{throws}\ H \ \mathsf{calls}\ S \quad \textit{and} \quad \Pi \vdash s : \Gamma \quad \textit{and} \quad \Pi \vdash h,$$

(1) if $(s, h) \vdash e \Downarrow v, h' \,\&\, w$*, then there exist some* $\Pi' \sqsupseteq \Pi$ *and* $(r \,\&\, U) \in T$ *such that*

$$\Pi' \vdash v : r \quad \textit{and} \quad \Pi' \vdash h' \quad \textit{and} \quad w \in U;$$

(2) if $(s,h) \vdash e \downarrow l, h' \,\&\, w$*, then there exist some* $\Pi' \sqsupseteq \Pi$ *and* $(r \,\&\, U) \in H$ *such that*

$$\Pi' \vdash l : r \quad \textit{and} \quad \Pi' \vdash h' \quad \textit{and} \quad w \in U;$$

(3) if $(s,h) \vdash e \Uparrow \,\&\, w$*, then* $w \in S(\eta)$*, where* $S(\eta) \subseteq \Sigma^{\leq\omega}$ *is obtained by substituting every* δ *in* S *by* $\eta(\delta)$*.*

Proof of Theorem 10(1)&(2). The operational semantics of terminating executions and exception throwing are defined mutually. So we prove (1) and (2) simultaneously. Similarly to the proof of [4, Theorem 5], we carry out the proof by induction over the sum of the depth of the derivation of the typing judgment and the depth of the derivation of the operational semantics judgment.

Here we prove the case when the last rule applied in the typing judgment is TRYCATCH as an example, and leave the other cases to the reader.

Suppose the typing judgment is $\Gamma \vdash \texttt{try}\ e_1\ \texttt{catch}(E\,x)\ e_2 : T'\ \mathsf{throws}\ H'\ \mathsf{calls}\ S'$. By rule inversion, we have $\Gamma \vdash e_1 : T\ \mathsf{throws}\ H\ \mathsf{calls}\ S$ where $H = r_1 \,\&\, U_1 \mid \ldots \mid r_n \,\&\, U_n$ and $\Gamma,\, x{:}\, r_i \vdash e_2 : T_i\ \mathsf{throws}\ H_i\ \mathsf{calls}\ S_i$ for any i such that $C \preceq E$ for some $C \in \mathit{Cls}(r_i)$, and we have also $T' = T \cup \bigcup_{i=1}^{n} U_i \cdot T_i$ and $H' = H/E \cup \bigcup_{i=1}^{n} U_i \cdot H_i$ and $S' = S \cup \bigcup_{i=1}^{n} U_i \cdot S_i$.

Assume $\Pi \vdash s : \Gamma$ and $\Pi \vdash h$.

If $(s,h) \vdash \texttt{try}\ e_1\ \texttt{catch}(E\,x)\ e_2 \Downarrow v, h' \,\&\, w$, there are two possibilities by rule inversion:

- If $(s,h) \vdash e_1 \Downarrow v, h' \,\&\, w$, then by induction hypothesis we have $\Pi' \sqsupseteq \Pi$ and $(r \,\&\, U) \in T$ such that $\Pi' \vdash v : r$ and $\Pi' \vdash h'$ and $w \in U$. We are done because $T \subseteq T'$.
- If $(s,h) \vdash e_1 \downarrow l, h_1 \,\&\, w_1$ and $\mathrm{classOf}_{h_1}(l) \preceq E$ and $(s[x \mapsto l], h_1) \vdash e_2 \Downarrow v, h_2 \,\&\, w_2$, then $h' = h_2$ and $w = w_1 w_2$. By induction hypothesis, we have $\Pi' \sqsupseteq \Pi$ and i such that $\Pi' \vdash l : r_i$ and $\Pi' \vdash h_1$ and $w_1 \in U_i$. Then we know $\mathrm{classOf}_{h_1}(l) \in \mathit{Cls}(r_i)$, and thus have $\Gamma,\, x{:}\, r_i \vdash e_2 : T_i\ \mathsf{throws}\ H_i\ \mathsf{calls}\ S_i$. By induction hypothesis again, we have $\Pi'' \sqsupseteq \Pi'$ and $(r \,\&\, U) \in T_i$ such that $\Pi'' \vdash v : r$ and $\Pi'' \vdash h_2$ and $w_2 \in U$. We are done because $w = w_1 w_2 \in U_i \cdot U$ and $(r \,\&\, U_i \cdot U) \in U_i \cdot T_i \subseteq T'$.

If $(s,h) \vdash \texttt{try}\ e_1\ \texttt{catch}(E\,x)\ e_2 \downarrow l, h' \,\&\, w$, there are also two possibilities by rule inversion:

- If $(s,h) \vdash e_1 \downarrow l, h' \,\&\, w$ and $\mathrm{classOf}_{h'}(l) \not\preceq E$, then by induction hypothesis, we have $\Pi' \sqsupseteq \Pi$ and i such that $\Pi' \vdash l : r_i$ and $\Pi' \vdash h'$ and $w \in U_i$. Recall that H/E is obtained by removing from H the pairs $(r_i \,\&\, U_i)$ such that all the classes allowed by r_i are below E. Because $\mathrm{classOf}_{h'}(l) \in \mathit{Cls}(r_i)$ but $\mathrm{classOf}_{h'}(l) \not\preceq E$, we know $(r_i, U_i) \in H/E$. Then we are done because $H/E \subseteq H'$.
- If $(s,h) \vdash e_1 \downarrow l_1, h_1 \,\&\, w_1$ and $\mathrm{classOf}_{h_1}(l_1) \preceq E$ and $(s[x \mapsto l_1], h_1) \vdash e_2 \downarrow l_2, h_2 \,\&\, w_2$, then $l = l_2$ and $h' = h_2$ and $w = w_1 w_2$. By induction hypothesis, we have $\Pi' \sqsupseteq \Pi$ and i such that $\Pi' \vdash l_1 : r_i$ and $\Pi' \vdash h_1$ and $w_1 \in U_i$. We know $\mathrm{classOf}_{h_1}(l) \in \mathit{Cls}(r_i)$, and thus have $\Gamma,\, x{:}\, r_i \vdash e_2 : T_i\ \mathsf{throws}\ H_i\ \mathsf{calls}\ S_i$. By induction hypothesis again, we have $\Pi'' \sqsupseteq \Pi'$ and $(r \,\&\, U) \in H_i$ such that $\Pi'' \vdash l_2 : r$ and $\Pi'' \vdash h_2$ and $w_2 \in U$. We are done because $w = w_1 w_2 \in U_i \cdot U$ and $(r \,\&\, U_i \cdot U) \in U_i \cdot H_i \subseteq H'$. ◀

Now we move the soundness of the infinitary effect analysis. Recall that any expression $S \in \mathcal{P}(\Sigma^*)\langle \mathit{Sig} \rangle$ is of the form $\bigcup_{\delta \in \Delta} A_\delta \cdot \delta$ for some $\Delta \subseteq \mathit{Sig}$. If we treat the signatures as variables then each expression gives a $\mathcal{P}(\Sigma^{\leq\omega})$-valued function, *i.e.*, for any assignment $\eta : \mathit{Sig} \to \mathcal{P}(\Sigma^{\leq\omega})$, we obtain a language $S(\eta) \subseteq \Sigma^{\leq\omega}$ by substituting all δ in S by $\eta(\delta)$. It is easy to show that if $S \subseteq S'$ then $S(\eta) \subseteq S'(\eta)$ for any η.

For the proof of Theorem 10(3), we will use the following approximations of the greatest solution: Let $\{S_\delta \mid \delta \in \mathit{Sig}\}$ be the call expressions encoded in M. We define for each $n \in \mathbb{N}$ an assignment $\eta_n : \mathit{Sig} \to \mathcal{P}(\Sigma^{\leq\omega})$ inductively by

$$\eta_0(\delta) = \Sigma^{\leq\omega} \qquad \eta_{n+1}(\delta) = S_\delta(\eta_n).$$

Let η be the greatest solution of $\{\delta = S_\delta \mid \delta \in \mathit{Sig}\}$. Then we have $\eta(\delta) = \bigcap_{i\in\mathbb{N}} \eta_i(\delta)$ for every $\delta \in \mathit{Sig}$. See *e.g.* [6] for the characterization of simultaneous fixed points. More generally, we have $\eta(\delta) = \bigcap_{i\in N} \eta_i(\delta)$ for any infinite subset $N \subseteq \mathbb{N}$.

We firstly show that the above approximations satisfy the soundness theorem in the following sense:

▶ **Lemma 11.** *Given a well-typed program w.r.t. (F, M), let η_n's be the approximations to the greatest solution as defined above. For any n, Γ, e, T, H, S, Π, s, h and w, if*

$$\Gamma \vdash e : T \ \mathsf{throws}\ H \ \mathsf{calls}\ S \quad \textit{and} \quad \Pi \vdash s : \Gamma \quad \textit{and} \quad \Pi \vdash h \quad \textit{and} \quad (s,h) \vdash e \Uparrow \& \, w$$

then $w \in S(\eta_n)$.

Proof. By induction on n and on the depth of the typing derivation.

When $n = 0$, it is trivial because η_0 assigns $\Sigma^{\leq\omega}$ to all variables.

Assume that the statement holds for n. To show that it holds for $n+1$, we perform an induction on the depth of the typing derivation.

If the last used rule of the typing derivation is SUB or WEAK, then we have the desired result from the induction hypothesis. If not, we perform a case distinction over the last rule used in the derivation of operational semantics judgment. The cases of x, $\mathtt{emit}(a)$, *null*, $\mathtt{new}^\ell\ C$ and $\mathtt{throw}\ e$ are ruled out because their execution cannot diverge. Here we check the following cases.

- $\Gamma \vdash \mathtt{let}\ x = e_1\ \mathtt{in}\ e_2 : \bigcup_{i=1}^n U_i \cdot T_i \ \mathsf{throws}\ H \cup \bigcup_{i=1}^n U_i \cdot H_i \ \mathsf{calls}\ S \cup \bigcup_{i=1}^n U_i \cdot S_i$
  By rule inversion, we have $\Gamma \vdash e_1 : r_1 \ \& \ U_1 \mid \ldots \mid r_n \ \& \ U_n \ \mathsf{throws}\ H \ \mathsf{calls}\ S$ and $\Gamma, x\colon r_i \vdash e_2 : T_i \ \mathsf{throws}\ H_i \ \mathsf{calls}\ S_i$ for all $i \in \{1, \ldots, n\}$.
  Assume $\Pi \vdash s : \Gamma$ and $\Pi \vdash h$ and $(s,h) \vdash \mathtt{let}\ x = e_1\ \mathtt{in}\ e_2 \Uparrow \& \, w$. By rule inversion, we have two possibilities:
  - If $(s,h) \vdash e_1 \Downarrow v_1, h_1 \ \& \ w_1$ and $(s[x \mapsto v_1], h_1) \vdash e_2 \Uparrow \& \, w_2$, then we have $w = w_1 w_2$. By Theorem 10(1), we have $\Pi' \sqsupseteq \Pi$ and i such that $\Pi' \vdash v_1 : r_i$ and $\Pi' \vdash h_1$ and $w_1 \in U_i$. Then we have $\Pi' \vdash s[x \mapsto v_1] : (\Gamma, x : r_i)$. By induction hypothesis, we have $w_2 \in S_i(\eta_{n+1})$, and thus $w = w_1 w_2 \in U_i \cdot S_i(\eta_{n+1}) \subseteq (S \cup \bigcup_{i=1}^n U_i \cdot S_i)(\eta_{n+1})$.
  - If $(s,h) \vdash e_1 \Uparrow \& \, w$, then we have $w \in S(\eta_{n+1}) \subseteq (S \cup \bigcup_{i=1}^n U_i \cdot S_i)(\eta_{n+1})$ by induction hypothesis.
- $\Gamma, x\colon r, \bar{y}\colon \bar{s} \vdash x^C.m(\bar{y}) : T \ \mathsf{throws}\ H \ \mathsf{calls}\ (C, r, m, \bar{s}) \quad$ where $(T, H, _) = M(C, r, m, \bar{s})$
  Assume $\Pi \vdash s : (\Gamma, x\colon r, \bar{y}\colon \bar{s})$ and $\Pi \vdash h$ and $(s,h) \vdash x.m(\bar{y}) \Uparrow \& \, w$. The goal is to show $w \in \eta_{n+1}(C, r, m, \bar{s})$.
  Let $s(x) = l$, $h(l) = (D, _)$ and $\mathit{mtable}(D, m) = (\bar{z}, e)$. We have $(s', h) \vdash e \Uparrow \& \, w$ where $s' = [\mathtt{this} \mapsto l] \cup [x_i \mapsto s(y_i)]_{i\in\{1,\ldots,|\bar{x}|\}}$ by rule inversion.
  We have $\Pi \vdash l : r$ from the assumption $\Pi \vdash s : (\Gamma, x\colon r, \bar{y}\colon \bar{s})$. By rule inversion, we have two possibilities: (1) If $r = \mathtt{Unknown}$, then we know $D \in \mathit{Cls}(r)$. (2) If $r = \Pi(l)$, we also have $D \in \mathit{Cls}(r)$ from the assumption $\Pi \vdash h$. So we always have $D \in \mathit{Cls}(r)$.
  Let $\Gamma' = \mathtt{this}\colon D, \bar{z}\colon \bar{s}$ and $(T', H', S') = M(D, r, m, \bar{s})$. Because the program is well-typed, we have $\Gamma' \vdash e : T' \ \mathsf{throws}\ H' \ \mathsf{calls}\ S'$.
  We have also $\Pi \vdash s' : \Gamma'$. Then by induction hypothesis, we have $w \in S'(\eta_n)$.
  We know $D \preceq C$. Because M is well-formed, we have $S' \subseteq S_{C,r,m,\bar{s}}$. Therefore, we have $w \in S_{C,r,m,\bar{s}}(\eta_n) = \eta_{n+1}(C, r, m, \bar{s})$ by the definition of the approximations.

We leave the remaining cases to the reader. ◀

▶ **Lemma 12.** *For any call expression S and infinite set $N \subseteq \mathbb{N}$, we have $\bigcap_{i\in N} S(\eta_i) \subseteq S(\eta)$.*

Proof. By induction on the number of signatures in S. ◀

The soundness theorem for the case of divergence follows from the above lemmas:

Proof of Theorem 10(3). Suppose $\Gamma \vdash e : T$ throws H calls S and $\Pi \vdash s : \Gamma$ and $\Pi \vdash h$ and $(s, h) \vdash e \Uparrow \& w$. By the above lemmas, we have $w \in \bigcap_{n\in\mathbb{N}} S(\eta_n) \subseteq S(\eta)$. ◀

Therefore, if a program is well-typed, then the effects encoded in the typings are correct:

▶ **Corollary 13** (Soundness). *Suppose that a program is well-typed w.r.t. (F, M) and that η is an infinitary effect typing for M. Then, for any $(C, r, m, \bar{s}) \in Sig$ with $M(C, r, m, \bar{s}) = (T, H, S)$ and $\eta(C, r, m, \bar{s}) = V$ and for any $x{:}\, r$ and $\bar{y}{:}\, \bar{s}$, we have*

1. *if $x.m(\bar{y})$ evaluates to a value and generates a trace w, then the value is in region r and $w \in U$ for some $(r \,\&\, U) \in T$;*
2. *if $x.m(\bar{y})$ throws an exception and generates a trace w, then the exception is in region r and $w \in U$ for some $(r \,\&\, U) \in H$; and*
3. *if $x.m(\bar{y})$ diverges and generates a trace w, then $w \in V$.*

5 Region-based Büchi Type and Effect System

The effect type system introduced in Section 4 is not yet suitable for practical use. Because effects are given as arbitrary languages, we have no direct way to compute greatest fixed points of equation systems over $\mathcal{P}(\Sigma^{\leq\omega})$. In this section, we introduce a system (Section 5.2) where effect annotations are taken from a *finite* structure to represent the languages over Σ (Section 5.1) using the methods from abstract interpretation [3]. In Section 5.3, we present a type inference algorithm that infers a class table for a given program, provided a standard Featherweight Java typing for the program is given.

5.1 Büchi Abstraction

Instead of $\mathcal{P}(\Sigma^*)$ and $\mathcal{P}(\Sigma^{\leq\omega})$, we work with a finite structure that abstracts the languages over Σ for effect annotations, over which we can compute fixed points.

▶ **Definition 14.** *Let $\mathcal{M}_*$ and $\mathcal{M}_{\leq\omega}$ be join-semilattices, that is, a partial order together with a join operation. We use the symbols $\sqsubseteq$, $\sqcup$ to refer to their generic structure of join-semilattices. We call $(\mathcal{M}_*, \mathcal{M}_{\leq\omega})$ a* Büchi algebra *if it is equipped with the following operations:*

- *a binary operation on $\mathcal{M}_*$, written as $\mathcal{A} \cdot \mathcal{B}$ for $\mathcal{A}, \mathcal{B} \in \mathcal{M}_*$,*
- *a mapping $\mathcal{M}_* \times \mathcal{M}_{\leq\omega} \to \mathcal{M}_{\leq\omega}$, also written as $\mathcal{A} \cdot \mathcal{V}$ for $\mathcal{A} \in \mathcal{M}_*$ and $\mathcal{V} \in \mathcal{M}_{\leq\omega}$, and*
- *an operation $(-)^\omega : \mathcal{M}_* \to \mathcal{M}_{\leq\omega}$,*

such that all the operations are monotone and the binary operations are associative, i.e.*, $\mathcal{A} \cdot (\mathcal{B} \cdot \mathcal{U}) = (\mathcal{A} \cdot \mathcal{B}) \cdot \mathcal{U}$ for any $\mathcal{A}, \mathcal{B} \in \mathcal{M}_*$ and $\mathcal{U} \in \mathcal{M}_* \cup \mathcal{M}_{\leq\omega}$.*

For instance, $\mathcal{P}(\Sigma^*)$ and $\mathcal{P}(\Sigma^{\leq\omega})$ form a Büchi algebra. They are complete lattices. But note that meets are not required here because set intersections are not needed in the type system. We are looking for a *finite* Büchi algebra $(\mathcal{M}_*, \mathcal{M}_{\leq\omega})$ that abstracts them.

Recall that a *Galois insertion* between partial orders A, C consists of monotone maps $\alpha : C \to A$ and $\gamma : C \to A$ such that, for all $a \in A$ and $c \in C$,

$$\alpha(\gamma(a)) = a \qquad \gamma(\alpha(c)) \sqsupseteq c.$$

Intuitively, the function α furnishes for each concrete value in C its *abstraction*, whereas γ *concretizes* the abstract values in A. Moreover, the abstraction of c has a concretization lying above c.

Now we are ready to introduce the finite structure needed for presenting the type system and the type inference algorithm.

▶ **Definition 15.** *A* Büchi abstraction *is a* finite *Büchi algebra* $(\mathcal{M}_*, \mathcal{M}_{\leq\omega})$ *equipped with Galois insertions* (α_*, γ_*) *between* $\mathcal{M}_*$ *and* $\mathcal{P}(\Sigma^*)$ *and* $(\alpha_{\leq\omega}, \gamma_{\leq\omega})$ *between* $\mathcal{M}_{\leq\omega}$ *and* $\mathcal{P}(\Sigma^{\leq\omega})$ *such that the abstraction functions* α_* *and* $\alpha_{\leq\omega}$ *are compatible with the algebraic structure:*

$$\alpha_*(A \cdot B) = \alpha_*(A) \cdot \alpha_*(B) \quad \alpha_{\leq\omega}(A \cdot V) = \alpha_*(A) \cdot \alpha_{\leq\omega}(V) \quad \alpha_{\leq\omega}(A^\omega) = \alpha_*(A)^\omega$$

for all $A, B \subseteq \Sigma^*$ *and* $V \subseteq \Sigma^{\leq\omega}$.

Readers may have expected more structures on a Büchi abstraction due to the analogy to $(\mathcal{P}(\Sigma^*), \mathcal{P}(\Sigma^{\leq\omega}))$. Some of them can be constructed. For instance, the least elements of $\mathcal{M}_*$ and $\mathcal{M}_{\leq\omega}$ are exactly the abstraction of the empty set. We define the finite iteration $(-)^*$ on $\mathcal{M}_*$ as the least fixed point $\mathcal{A}^* = \mathsf{lfp}(\lambda X.\alpha_*(\{\varepsilon\}) \sqcup \mathcal{A} \cdot X)$, and then have $\alpha_*(A^*) = (\alpha_*(A))^*$. Moreover, we have the following properties of the Büchi abstraction.

▶ **Proposition 16.** **1.** *The abstraction functions* α_* *and* $\alpha_{\leq\omega}$ *preserve joins,* i.e., $\alpha_*(A \cup B) = \alpha_*(A) \sqcup \alpha_*(B)$ *and* $\alpha_{\leq\omega}(U \cup V) = \alpha_{\leq\omega}(U) \sqcup \alpha_{\leq\omega}(V)$.

2. *The concretization functions* γ_* *and* $\gamma_{\leq\omega}$ *may not preserve joins or concatenations, but they satisfy*

$$\gamma_*(\mathcal{A} \sqcup \mathcal{B}) \supseteq \gamma_*(\mathcal{A}) \cup \gamma_*(\mathcal{B}) \qquad \gamma_*(\mathcal{A} \cdot \mathcal{B}) \supseteq \gamma_*(\mathcal{A}) \cdot \gamma_*(\mathcal{B})$$
$$\gamma_{\leq\omega}(\mathcal{U} \sqcup \mathcal{V}) \supseteq \gamma_{\leq\omega}(\mathcal{U}) \cup \gamma_{\leq\omega}(\mathcal{V}) \qquad \gamma_{\leq\omega}(\mathcal{A} \cdot \mathcal{V}) \supseteq \gamma_*(\mathcal{A}) \cdot \gamma_{\leq\omega}(\mathcal{V}).$$

3. *The concatenation operations are distributive over joins,* i.e., $(\mathcal{A} \sqcup \mathcal{B}) \cdot \mathcal{V} = \mathcal{A} \cdot \mathcal{V} \sqcup \mathcal{B} \cdot \mathcal{V}$ *and* $\mathcal{A} \cdot (\mathcal{U} \sqcup \mathcal{V}) = \mathcal{A} \cdot \mathcal{U} \sqcup \mathcal{A} \cdot \mathcal{V}$.

4. *Lastly, we have* $\alpha_{\leq\omega}(\gamma_*(\mathcal{A})^\omega) = \mathcal{A}^\omega = \mathcal{A} \cdot \mathcal{A}^\omega$.

Note that, differing from finite iteration, here we do not define (or require) ω-iteration to be the greatest fixed point. We have only $\mathcal{A}^\omega \sqsubseteq \mathsf{gfp}(\lambda X.\mathcal{A} \cdot X)$ in general, because $\mathcal{A}^\omega$ is a fixed point of $\lambda X.\mathcal{A} \cdot X$ by Proposition 16(4). See Example 17 where a ω-iteration is strictly smaller than the greatest fixed point.

▶ **Example 17.** Let the alphabet be $\Sigma = \{a\}$. Now we give a concrete example for a Büchi abstraction that distinguishes empty from non-empty words: $\mathcal{M}_* = \{\emptyset, \{\varepsilon\}, a^+, a^*\}$ and $\mathcal{M}_{\leq\omega} = \{\emptyset, \{\varepsilon\}, a^+, a^*, a^\omega, \{\varepsilon\} \cup a^\omega, a^+ \cup a^\omega, a^* \cup a^\omega\}$. For both, $\sqcup$ and $\sqsubseteq$ are set union and inclusion. All other operations are given by the evident operations on languages. The abstraction function α_* is determined by $\alpha_*(\{\varepsilon\}) = \{\varepsilon\}$ and $\alpha_*(X) = a^+$ for any $X \subseteq a^+$. All other values are determined by preservation of least element and joins. Similarly, $\alpha_{\leq\omega}$ is determined by $\alpha_{\leq\omega}(\{\varepsilon\}) = \{\varepsilon\}$, $\alpha_{\leq\omega}(X) = a^+$ for any $X \subseteq a^+$ and $\alpha_{\leq\omega}(a^\omega) = a^\omega$. The concretion functions γ_* and $\gamma_{\leq\omega}$ are both the identity.

We note that the greatest solution of the equation $X = a^+ \cdot X$ over $\mathcal{M}_{\leq\omega}$ is $a^+ \cup a^\omega$ just by direct calculation. When calculated in $\mathcal{P}(\Sigma^{\leq\omega})$, the greatest solution is a^ω which is the ω-iteration of a^+ over $\mathcal{M}_{\leq\omega}$. ◀

As will become clear, a Büchi abstraction is sufficient for presenting the type system and the type inference algorithm. Hofmann and Chen [10, 9] construct a Büchi abstraction from an *extended Büchi automaton* $\mathfrak{A}$ whose language is the union of the language of $\mathfrak{A}$

when understood as a traditional nondeterministic finite automaton and the language of $\mathfrak{A}$ when understood as a traditional Büchi automaton. A programming guideline is simply a set of finite and infinite traces, and thus can be represented as an extended Büchi automaton. Moreover, the Büchi abstraction of Hofmann and Chen is faithful with respect to acceptance by the automaton, which makes it suitable for representing effects in our type system. However, the development of the system (Section 5.2) and the type inference algorithm (Section 5.3) are *independent* of the construction of the Büchi abstraction. Readers should be able to access them without probing into the construction of Hofmann and Chen.

Similarly to the development in the previous section, we work with formal expressions $\mathcal{M}_*\langle \mathit{Reg}\rangle$ and $\mathcal{M}_*\langle \mathit{Sig}\rangle$ in the type system. We define the order $\sqsubseteq$ and the operations of concatenation $(\cdot)$ and join $(\sqcup)$ on expressions in the same way as in Definition 2. To relate to the previous system, we extend the abstraction and concretization functions to expressions: Let K be either Reg or Sig. Given $\mathcal{T} \in \mathcal{M}_*\langle K\rangle$, we concretize it to $\gamma(\mathcal{T}) \in \mathcal{P}(\Sigma^*)\langle K\rangle$ by defining $\gamma(\mathcal{T})(x) = \gamma_*(\mathcal{T}(x))$. Similarly, we abstract $T \in \mathcal{P}(\Sigma^*)\langle K\rangle$ to $\alpha(T) \in \mathcal{M}_*\langle K\rangle$. By the definition and properties of Büchi abstraction, we have:

$$\alpha(\gamma(\mathcal{T})) = \mathcal{T} \qquad \alpha(T \cup T') = \alpha(T) \sqcup \alpha(T') \qquad \alpha(U \cdot T) = \alpha_*(U) \cdot \alpha(T)$$
$$\gamma(\alpha(T)) \supseteq T \qquad \gamma(\mathcal{T} \sqcup \mathcal{T}') \supseteq \gamma(\mathcal{T}) \cup \gamma(\mathcal{T}') \qquad \gamma(\mathcal{U} \cdot \mathcal{T}) \supseteq \gamma_*(\mathcal{U}) \cdot \gamma(\mathcal{T}).$$

5.2 Büchi Type and Effect System

In this section, we assume a programming guideline given by the language $\mathfrak{A}$ and a Büchi abstraction $(\mathcal{M}_*, \mathcal{M}_{\leq\omega})$ which is faithful w.r.t. $\mathfrak{A}$, *i.e.*, $\gamma_{\leq\omega}(\alpha_{\leq\omega}(\mathfrak{A})) = \mathfrak{A}$.

We now present an algorithmic type system with effects being elements of $\mathcal{M}_*$ and $\mathcal{M}_{\leq\omega}$. The development is similar to the one in the previous section. We remove the subtyping rule and replace sets of traces by their abstractions in the type system. Thanks to the finite abstraction, we can compute greatest solutions to capture the possible non-terminating traces of a program. However, the abstraction may not preserve greatest fixed points in general. To make the analysis more precise, we concretize the abstract call expressions when defining infinitary effect typing. And we provide a simple algorithm to compute the infinitary effects that abstract the greatest fixed point of the concretized expressions.

In a *Büchi class table* $(F, M_{\mathfrak{A}})$, the field typing $F : \mathit{Cls} \times \mathit{Reg} \times \mathit{Fld} \rightharpoonup \mathcal{P}(\mathit{Reg})$ is the same as the one in the previous setting, while the method typing $M_{\mathfrak{A}} : \mathit{Sig} \to \mathcal{M}_*\langle \mathit{Reg}\rangle \times \mathcal{M}_*\langle \mathit{Reg}\rangle \times \mathcal{M}_*\langle \mathit{Sig}\rangle$ gives effect expressions with abstract sets of traces. We require $(F, M_{\mathfrak{A}})$ to be well-formed as in the previous setting.

A *Büchi typing judgment* takes the form $\Gamma \vdash_{\mathfrak{A}} e : \mathcal{T}$ throws $\mathcal{H}$ calls $\mathcal{S}$ where $\mathcal{T} : \mathcal{M}_*\langle \mathit{Reg}\rangle$ lists the possible regions and terminating effects of e, $\mathcal{H} : \mathcal{M}_*\langle \mathit{Reg}\rangle$ lists the regions and effects of the exceptions that e may throw, and $\mathcal{S} : \mathcal{M}_*\langle \mathit{Sig}\rangle$ specifies the information about the calls that e may make as in the previous system, except that effects are given by the Büchi abstraction. The typing rules are listed in Figure 2. They are the same to those in the previous setting. The only difference is that the effect expressions are interpreted over the Büchi algebra rather than languages: Any singleton $\{a\}$ becomes its abstraction $\alpha_*(\{a\})$ and the operations of join and concatenation on languages are replaced by the same operations from the Büchi algebra. In particular, there is no subtyping rule. But we keep the WEAK rule for the consideration of efficiency (see the discussion in Section 6).

A program $(\prec, \mathit{fields}, \mathit{methods}, \mathit{mtable})$ is *well-typed* w.r.t. a Büchi class table $(F, M_{\mathfrak{A}})$ if for every method signature $(C, r, m, \bar{s}) \in \mathit{Sig}$ with $C \in \mathit{Cls}(r)$ and $\mathit{mtable}(C, m) = (\bar{x}, e)$, the typing judgment $\mathtt{this}\colon r,\ \bar{x}\colon \bar{s} \vdash_{\mathfrak{A}} e : \mathcal{T}$ throws $\mathcal{H}$ calls $\mathcal{S}$ is derivable for some expressions $(\mathcal{T}, \mathcal{H}, \mathcal{S}) \sqsubseteq M_{\mathfrak{A}}(C, r, m, \bar{s})$. Differing from Definition 5, here the method body has a type

$$\textsc{weak}\ \frac{\Gamma \vdash_{\mathfrak{A}} e : (r \,\&\, \mathcal{U}) \sqcup \mathcal{T}\ \mathsf{throws}\ \mathcal{H}\ \mathsf{calls}\ \mathcal{S}}{\Gamma \vdash_{\mathfrak{A}} e : (\mathtt{Unknown} \,\&\, \mathcal{U}) \sqcup \mathcal{T}\ \mathsf{throws}\ \mathcal{H}\ \mathsf{calls}\ \mathcal{S}}$$

$$\textsc{prim}\ \frac{}{\Gamma \vdash_{\mathfrak{A}} \mathtt{emit}(a) : \mathtt{Null} \,\&\, \alpha_*(\{a\})\ \mathsf{throws}\ \emptyset\ \mathsf{calls}\ \emptyset}$$

$$\textsc{var}\ \frac{}{\Gamma,\, x{:}\, r \vdash_{\mathfrak{A}} x : r \,\&\, \alpha_*(\{\varepsilon\})\ \mathsf{throws}\ \emptyset\ \mathsf{calls}\ \emptyset}$$

$$\textsc{if}\ \frac{\begin{array}{c}\Gamma,\, x{:}\, r,\, y{:}\, s \vdash_{\mathfrak{A}} e_1 : \mathcal{T}_1\ \mathsf{throws}\ \mathcal{H}_1\ \mathsf{calls}\ \mathcal{S}_1 \\ \Gamma,\, x{:}\, r,\, y{:}\, s \vdash_{\mathfrak{A}} e_2 : \mathcal{T}_2\ \mathsf{throws}\ \mathcal{H}_2\ \mathsf{calls}\ \mathcal{S}_2\end{array}}{\Gamma,\, x{:}\, r,\, y{:}\, s \vdash_{\mathfrak{A}} \mathtt{if}\ x = y\ \mathtt{then}\ e_1\ \mathtt{else}\ e_2 : \mathcal{T}_1 \sqcup \mathcal{T}_2\ \mathsf{throws}\ \mathcal{H}_1 \sqcup \mathcal{H}_2\ \mathsf{calls}\ \mathcal{S}_1 \sqcup \mathcal{S}_2}$$

$$\textsc{else}\ \frac{\mathit{disjoint}(r, s) \qquad \Gamma,\, x{:}\, r,\, y{:}\, s \vdash_{\mathfrak{A}} e_2 : \mathcal{T}_2\ \mathsf{throws}\ \mathcal{H}_2\ \mathsf{calls}\ \mathcal{S}_2}{\Gamma,\, x{:}\, r,\, y{:}\, s \vdash_{\mathfrak{A}} \mathtt{if}\ x = y\ \mathtt{then}\ e_1\ \mathtt{else}\ e_2 : \mathcal{T}_2\ \mathsf{throws}\ \mathcal{H}_2\ \mathsf{calls}\ \mathcal{S}_2}$$

$$\textsc{let}\ \frac{\begin{array}{c}\Gamma \vdash_{\mathfrak{A}} e_1 : r_1 \,\&\, \mathcal{U}_1 \mid \ldots \mid r_n \,\&\, \mathcal{U}_n\ \mathsf{throws}\ \mathcal{H}\ \mathsf{calls}\ \mathcal{S} \\ \Gamma,\, x{:}\, r_i \vdash_{\mathfrak{A}} e_2 : \mathcal{T}_i\ \mathsf{throws}\ \mathcal{H}_i\ \mathsf{calls}\ \mathcal{S}_i\end{array}}{\Gamma \vdash_{\mathfrak{A}} \mathtt{let}\ x = e_1\ \mathtt{in}\ e_2 : \bigsqcup_{i=1}^{n} \mathcal{U}_i \cdot \mathcal{T}_i\ \mathsf{throws}\ \mathcal{H} \sqcup \bigsqcup_{i=1}^{n} \mathcal{U}_i \cdot \mathcal{H}_i\ \mathsf{calls}\ \mathcal{S} \sqcup \bigsqcup_{i=1}^{n} \mathcal{U}_i \cdot \mathcal{S}_i}$$

$$\textsc{null}\ \frac{}{\Gamma \vdash_{\mathfrak{A}} \mathtt{null} : \mathtt{Null} \,\&\, \alpha_*(\{\varepsilon\})\ \mathsf{throws}\ \emptyset\ \mathsf{calls}\ \emptyset}$$

$$\textsc{new}\ \frac{}{\Gamma \vdash_{\mathfrak{A}} \mathtt{new}^{\ell}\ C : \mathtt{CreatedAt}(\ell) \,\&\, \alpha_*(\{\varepsilon\})\ \mathsf{throws}\ \emptyset\ \mathsf{calls}\ \emptyset}$$

$$\textsc{cast}\ \frac{\Gamma \vdash_{\mathfrak{A}} e : \mathcal{T}\ \mathsf{throws}\ \mathcal{H}\ \mathsf{calls}\ \mathcal{S}}{\Gamma \vdash_{\mathfrak{A}} (D)\, e : \mathcal{T}\ \mathsf{throws}\ \mathcal{H}\ \mathsf{calls}\ \mathcal{S}}$$

$$\textsc{get}\ \frac{\mathcal{T} = \bigsqcup\{s \,\&\, \alpha_*(\{\varepsilon\}) \mid s \in F(C, r, f)\}}{\Gamma,\, x{:}\, r \vdash_{\mathfrak{A}} x^C.f : \mathcal{T}\ \mathsf{throws}\ \emptyset\ \mathsf{calls}\ \emptyset}$$

$$\textsc{set}\ \frac{s \in F(C, r, f)}{\Gamma,\, x{:}\, r,\, y{:}\, s \vdash_{\mathfrak{A}} x^C.f := y : s \,\&\, \alpha_*(\{\varepsilon\})\ \mathsf{throws}\ \emptyset\ \mathsf{calls}\ \emptyset}$$

$$\textsc{call}\ \frac{(\mathcal{T}, \mathcal{H}, _) = M_{\mathfrak{A}}(C, r, m, \bar{s})}{\Gamma,\, x{:}\, r,\, \bar{y}{:}\, \bar{s} \vdash_{\mathfrak{A}} x^C.m(\bar{y}) : \mathcal{T}\ \mathsf{throws}\ \mathcal{H}\ \mathsf{calls}\ (C, r, m, \bar{s})}$$

$$\textsc{throw}\ \frac{\Gamma \vdash_{\mathfrak{A}} e : \mathcal{T}\ \mathsf{throws}\ \mathcal{H}\ \mathsf{calls}\ \mathcal{S}}{\Gamma \vdash_{\mathfrak{A}} \mathtt{throw}\ e : \emptyset\ \mathsf{throws}\ \mathcal{T} \sqcup \mathcal{H}\ \mathsf{calls}\ \mathcal{S}}$$

$$\textsc{trycatch}\ \frac{\begin{array}{c}\Gamma \vdash_{\mathfrak{A}} e_1 : \mathcal{T}\ \mathsf{throws}\ \mathcal{H}\ \mathsf{calls}\ S \qquad \mathcal{H} = r_1 \,\&\, \mathcal{U}_1 \mid \ldots \mid r_n \,\&\, \mathcal{U}_n \\ \Gamma,\, x{:}\, r_i \vdash_{\mathfrak{A}} e_2 : \mathcal{T}_i\ \mathsf{throws}\ \mathcal{H}_i\ \mathsf{calls}\ \mathcal{S}_i \text{ and } C \preceq E \text{ for some } C \in \mathit{Cls}(r_i)\end{array}}{\begin{array}{l}\Gamma \vdash_{\mathfrak{A}} \mathtt{try}\ e_1\ \mathtt{catch}(E\, x)\ e_2 : \mathcal{T} \sqcup \bigsqcup_{i=1}^{n} \mathcal{U}_i \cdot \mathcal{T}_i\ \mathsf{throws}\ \mathcal{H}/E \sqcup \bigsqcup_{i=1}^{n} \mathcal{U}_i \cdot \mathcal{H}_i \\ \qquad\qquad \mathsf{calls}\ \mathcal{S} \sqcup \bigsqcup_{i=1}^{n} \mathcal{U}_i \cdot \mathcal{S}_i\end{array}}$$

Figure 2 Region-based type and effect system

below the one encoded in $M_{\mathfrak{A}}$. This is because $M_{\mathfrak{A}}$ is invariant w.r.t superclass and thus

may contain more than the actual effects of the method, but the Büchi type system does not allow subtyping.

Given a Büchi method typing $M_{\mathfrak{A}}$, we obtain a method typing $M^{\gamma}_{\mathfrak{A}}$ with concretized effects by defining for $\delta \in \mathrm{dom}(M_{\mathfrak{A}})$ with $M_{\mathfrak{A}}(\delta) = (\mathcal{T}, \mathcal{H}, \mathcal{S})$:

$$M^{\gamma}_{\mathfrak{A}}(\delta) = (\gamma(\mathcal{T}), \gamma(\mathcal{H}), \gamma(\mathcal{S})).$$

We relate the Büchi type system to the type system introduced in Section 4 as follows.

▶ **Lemma 18.** *If* $\Gamma \vdash_{\mathfrak{A}} e : \mathcal{T}$ throws $\mathcal{H}$ calls $\mathcal{S}$ *is derivable w.r.t. a Büchi class table* $(F, M_{\mathfrak{A}})$*, then so is* $\Gamma \vdash e : \gamma(\mathcal{T})$ throws $\gamma(\mathcal{H})$ calls $\gamma(\mathcal{S})$ *w.r.t. the corresponding class table* $(F, M^{\gamma}_{\mathfrak{A}})$*.*

The following is a direct consequence of Lemma 18.

▶ **Theorem 19.** *If a program* P *is well-typed w.r.t. a Büchi class table* $(F, M_{\mathfrak{A}})$*, then it is well-typed w.r.t. the corresponding class table* $(F, M^{\gamma}_{\mathfrak{A}})$*.*

Moreover, we can compute the infinitary effect typing over the Büchi abstraction.

▶ **Theorem 20.** *Given a Büchi method typing* $M_{\mathfrak{A}}$*, there is an assignment* $\eta_{\mathfrak{A}} : \mathit{Sig} \to \mathcal{M}_{\leq\omega}$ *that abstracts the infinitary effect typing* $\eta_{\gamma} : \mathit{Sig} \to \mathcal{P}(\Sigma^{\leq\omega})$ *for* $M^{\gamma}_{\mathfrak{A}}$*,* i.e.*, for all* $\delta \in \mathit{Sig}$ *we have* $\eta_{\mathfrak{A}}(\delta) = \alpha(\eta_{\gamma}(\delta))$ *where* η_{γ} *is the greatest solution of* $\{\delta = \gamma(\mathcal{S}_{\delta}) \mid \delta \in \mathit{Sig}\}$ *with* $\mathcal{S}_{\delta}$ *the call expression in* $M_{\mathfrak{A}}(\delta)$*. We call such an assignment* $\eta_{\mathfrak{A}}$ *a* Büchi infinitary effect typing*.*

Proof. Let $\mathcal{S}_{\delta}$ be the call expressions in $M_{\mathfrak{A}}(\delta)$. We solve $\{\delta = \mathcal{S}_{\delta} \mid \delta \in \mathit{Sig}\}$ as follows: Recall that each expression is of the form $\bigsqcup_{\delta\in\Delta} \mathcal{A}_{\delta} \cdot \delta$ for some $\Delta \subseteq \mathit{Sig}$. Viewing as an $\mathcal{M}_{\leq\omega}$-valued function, each $\mathcal{S}_{\delta}$ can be rewritten to an equivalent form $\mathcal{A}_{\delta} \cdot \delta \sqcup \mathcal{F}_{\delta}$ where the function $\mathcal{F}_{\delta}$ does not contain δ, by the fact that the join operation is commutative. We repeat the following steps for each $\delta \in \mathit{Sig}$:

1. Rewrite the right-hand side of the equation for δ to $\mathcal{A}_{\delta} \cdot \delta \sqcup \mathcal{F}_{\delta}$.
2. Set $\delta = \mathcal{A}^{*}_{\delta} \cdot \mathcal{F}_{\delta} \sqcup \mathcal{A}^{\omega}_{\delta}$.
3. Substitute δ by $\mathcal{A}^{*}_{\delta} \cdot \mathcal{F}_{\delta} \sqcup \mathcal{A}^{\omega}_{\delta}$ on the right-hand side of all the other equations.

One variable is eliminated on the right-hand side of the equations after each iteration. In the end we solve the system and obtain an assignment function $\eta_{\mathfrak{A}} : \mathit{Sig} \to \mathcal{M}_{\leq\omega}$.

It abstracts the greatest solution η_{γ} of the system $\{\delta = \gamma(\mathcal{S}_{\delta}) \mid \delta \in \mathit{Sig}\}$ of concretized equations. For instance, consider a singleton equation system $\{X = \mathcal{A} \cdot X \sqcup \mathcal{B}\}$ whose solution is $\eta_{\mathfrak{A}}(X) = \mathcal{A}^{*} \cdot \mathcal{B} \sqcup \mathcal{A}^{\omega}$ according to the above algorithm. The greatest solution of its concretization $\{X = \gamma_{*}(\mathcal{A}) \cdot X \cup \gamma_{\leq\omega}(\mathcal{B})\}$ is $\eta_{\gamma}(X) = \gamma_{*}(\mathcal{A})^{*} \cdot \gamma_{\leq\omega}(\mathcal{B}) \cup \gamma_{*}(\mathcal{A})^{\omega}$. It is not hard to prove $\eta_{\mathfrak{A}}(X) = \alpha_{\leq\omega}(\eta_{\gamma}(X))$ using Proposition 16. One can easily generalize the proof to systems with multiple equations. We refer the reader to *e.g.* [6] for the characterization of simultaneous fixed points. ◀

▶ Remark 21. One may expect a Büchi infinitary effect typing to be the greatest solution of $\{\delta = \mathcal{S}_{\delta} \mid \delta \in \mathit{Sig}\}$ without concretization. However, the Büchi abstraction may not preserve greatest fixed points, which may lead to a loss in precision. Consider the example of a single method `f` with body $\mathtt{emit}(a); \mathtt{this.f}()$. The equation system is $\{\delta_{\mathtt{f}} = \alpha_{*}(\{a\}) \cdot \delta_{\mathtt{f}}\}$. If we solve it in the Büchi abstraction from Example 17, then we get the solution $\eta(\delta_{\mathtt{f}}) = a^{+} \cup a^{\omega}$, as noted there. However, if we concretize this equation and solve over sets, then we get the solution $\eta_{\gamma}(\delta_{\mathtt{f}}) = a^{\omega}$. Therefore, we choose a better approximation of the infinite traces and formulate the Büchi infinitary effect typing as the abstraction of the greatest fixed point of the concretized call expressions.

Therefore, the soundness of the Büchi type system follows directly from the soundness of the previous type system (Corollary 13).

▶ **Corollary 22** (Soundness). *Suppose that a program is well-typed w.r.t.* $(F, M_{\mathfrak{A}})$ *and that* $\eta_{\mathfrak{A}}$ *is a Büchi infinitary effect typing for* $M_{\mathfrak{A}}$*. Then, for any method signature* $(C, r, m, \bar{s})$ *with* $M_{\mathfrak{A}}(C, r, m, \bar{s}) = (\mathcal{T}, \mathcal{H}, \mathcal{S})$ *and* $\eta_{\mathfrak{A}}(C, r, m, \bar{s}) = \mathcal{V}$*, and for any* $x{:}\, r$ *and* $\bar{y}{:}\, \bar{s}$*, we have*

1. *if* $x.m(\bar{y})$ *evaluates to a value and generates a trace* w*, then the value is in region* r *and* $w \in \gamma_*(\mathcal{U})$ *for some* $(r \,\&\, \mathcal{U}) \in \mathcal{T}$*;*
2. *if* $x.m(\bar{y})$ *throws an exception and generates a trace* w*, then the exception is in region* r *and* $w \in \gamma_*(\mathcal{U})$ *for some* $(r \,\&\, \mathcal{U}) \in \mathcal{H}$*; and*
3. *if* $x.m(\bar{y})$ *diverges and generates a trace* w*, then* $w \in \gamma_{\leq\omega}(\mathcal{V})$*.*

To certify that all the traces of a given method are allowed by the language $\mathfrak{A}$, we only need to check if all its effects stored in the typings $M_{\mathfrak{A}}$ and $\eta_{\mathfrak{A}}$ are below the abstraction of $\mathfrak{A}$. Come back to the example of a server given in the Introduction. Let $\mathfrak{A}$ be the guideline consisting of traces where each event *access* is immediately preceded by *authcheck*. It can be represented by a Büchi automaton. Based on the automaton, we employ the construction of Hofmann and Chen [9] to get a Büchi abstraction. Checking whether the non-terminating effect of `serve` is below $\mathfrak{A}$ amounts to checking whether it is accepted by the Büchi automaton. In this example, this will be the case and `serve` is verified to adhere to the guideline.

5.3 Type Inference Algorithm

Given a program P that is well-typed w.r.t. a given standard Featherweight Java class table (F_0, M_0), we extend the inference algorithm of [4, Appendix F] to construct a Büchi class table $(F, M_{\mathfrak{A}})$ with respect to which P is well-typed.

We follow the typing rules to construct a procedure typeff that maps an environment $\Gamma : \mathit{Var} \rightharpoonup \mathit{Reg}$ and a term $e \in \mathit{Expr}$ to formal expressions $\mathcal{T}, \mathcal{H} \in \mathcal{M}_*\langle \mathit{Reg} \rangle$ and $\mathcal{S} \in \mathcal{M}_*\langle \mathit{Sig} \rangle$, if e is typable in Γ, such that $\Gamma \vdash_{\mathfrak{A}} e : \mathcal{T}$ throws $\mathcal{H}$ calls $\mathcal{S}$ is derivable. In particular, typeff may update the field typing F, which happens in the case of a field-set statement $x^C.f = y$. The SET rule requires that the region of y must be in those allowed for the field f by F. If this condition is not satisfied, then typeff updates F by joining the offending entry with the region of y. We then use typeff to construct a Büchi class table. Rule WEAK does not have to be applied, but it can be used to trade precision of analysis for efficiency of the inference algorithm. See the discussion in Section 6.

In brief, the class table is computed as a least fixed point using the typeff procedure. We start with a table $(F, M_{\mathfrak{A}})$ with the possibly "lowest" entries, by lifting F_0 with $\{\texttt{Null}\}$ and M_0 with the empty expression. For each method signature, we "increase" the corresponding entry in $M_{\mathfrak{A}}$ if it differs from the result given by calling typeff with the method body. Note that typeff may update F as discussed above. Then we ensure that the table is well-formed. We repeat this until the table cannot be updated further. Because there are only finitely many regions and abstract effects, we will reach the fixed point after some iterations. The resulting table $(F, M_{\mathfrak{A}})$ encodes the terminating effects and the formal expressions for computing the non-terminating effects of the program.

▶ **Theorem 23.** *Let* P *be a program that is well-typed w.r.t. some standard FJ class table* (F_0, M_0)*. Then the above algorithm gives a Büchi class table* $(F, M_{\mathfrak{A}})$ *with respect to which* P *is well-typed.*

6 Conclusion and Discussion

This paper presents a framework to verify if a given Featherweight Java program adheres to a given programming guideline. Let $\mathfrak{A}$ be a language representing the guideline. Suppose a Büchi abstraction that is faithful w.r.t. $\mathfrak{A}$ is given, *e.g.* the one of Hofmann and Chen [10]. From the standard Java types of the fields and methods, we construct a Büchi class table $(F, M_{\mathfrak{A}})$ with respect to which the program is well-typed. The table $M_{\mathfrak{A}}$ and its infinitary effect typing $\eta_{\mathfrak{A}}$ provide the terminating and non-terminating effects to each method signature. If the effects of the methods are below the abstraction of $\mathfrak{A}$, then any event trace generated by the program is allowed by $\mathfrak{A}$ and thus the program adheres to the guideline.

We work along the line of typing Java programs with regions and effects [8, 2, 4]. In stead of refining the standard Featherweight Java type system as in the previous work, we introduce a flow type system where class information from the standard type system can be omitted. Our approach to region typing is in the spirit of Microsoft's TypeScript [13] and Facebook's Flow [5]. Moving to a flow type system has the benefit of simplifying the type system and making the choice of regions more flexible. In further work, we intend to work with richer domains of regions, *e.g.* with lattice structure, to capture more detailed information about objects. Our system is similar to the session type and effect system for PCF studied in [14]. But ours can capture also effects of non-terminating programs.

The idea of a Java type system with Büchi effects has been sketched in [9, 10]. But its meta-theoretic properties like soundness and the correctness of type inference were not investigated. Here we develop this idea fully and establish the meta-theory. Furthermore, we make the type system independent from the automata-theoretic constructions of [9, 10] by capturing the necessary structure abstractly in terms of a Büchi abstraction. Büchi automata have been employed in model checking and testing for infinite executions of programs [11, 1, 16]. Our approach is entirely based on type systems and abstract interpretation.

Comparing to the previous work [4, 9], our approach enhances the precision of analysis by capturing the effect for each region where a value may locate. The cost of this improvement is that the type inference is less efficient. For instance, to compute the effect of the expression $\texttt{let}\ x = e_1\ \texttt{in}\ e_2$, we need to infer e_2 as many times as the number of possible regions where the value of e_1 may locate, while in the systems of [4, 9] e_2 is inferred only once. However, our framework is actually very flexible. If we apply the WEAK rule in every step of the typing derivation, then we essentially work with an effect type system without regions, because in the end only the region `Unknown` remains in the result and all effects of a value are merged. If we extend regions with a join operation $\texttt{Union}(r_1, r_2)$ and a corresponding lattice structure, and change the WEAK rule to allow merging $r_1\ \&\ U_1 \mid r_2\ \&\ U_2$ to $\texttt{Union}(r_1, r_2) \mid (U_1 \cup U_2)$, then we can recover the systems of [4, 9]. In addition, we can reduce the size of the method table in an implementation by letting $M(C, \texttt{Union}(r_1, r_2), m, \bar{s}) = M(C, r_1, m, \bar{s}) \sqcup M(C, r_2, m, \bar{s})$ and storing only the latter two entries. Moreover, one may come up with certain strategy about when to apply the WEAK rule in the inference algorithm to balance efficiency and precision. It is our future work to investigate such directions.

References

1 Damián Adalid, Alberto Salmerón, María del Mar Gallardo, and Pedro Merino. Using SPIN for automated debugging of infinite executions of Java programs. *Journal of Systems and Software*, 90:61–75, 2014. doi:https://doi.org/10.1016/j.jss.2013.10.056.

2 Lennart Beringer, Robert Grabowski, and Martin Hofmann. Verifying pointer and string analyses with region type systems. *Computer Languages, Systems & Structures*, 39(2):49–65, 2013. doi:10.1016/j.cl.2013.01.001.

3 Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In Robert M. Graham, Michael A. Harrison, and Ravi Sethi, editors, *Principles of Programming Languages (POPL 1977)*, pages 238–252, 1977. doi:10.1145/512950.512973.

4 Serdar Erbatur, Martin Hofmann, and Eugen Zălinescu. Enforcing programming guidelines with region types and effects. In Bor-Yuh Evan Chang, editor, *Programming Languages and Systems (APLAS 2017)*, volume 10695 of *Lecture Notes in Computer Science*, pages 85–104. Springer, Cham, 2017. doi:10.1007/978-3-319-71237-6_5.

5 Facebook. Flow - A static type checker for JavaScript. URL: https://flow.org.

6 Carsten Fritz. Some fixed point basics. In Erich Grädel, Wolfgang Thomas, and Thomas Wilke, editors, *Automata Logics, and Infinite Games*, volume 2500 of *Lecture Notes in Computer Science*, pages 359–364. Springer, Berlin, Heidelberg, 2002. doi:10.1007/3-540-36387-4_20.

7 James Gosling, Bill Joy, Guy Steele, Gilad Bracha, and Alex Buckley. *The Java Language Specification*. Oracle America, Inc., Java SE 14 edition, 2020. URL: http://docs.oracle.com/javase/specs/jls/se14/jls14.pdf.

8 Robert Grabowski, Martin Hofmann, and Keqin Li. Type-based enforcement of secure programming guidelines — code injection prevention at SAP. In G. Barthe, A. Datta, and S. Etalle, editors, *Formal Aspects of Security and Trust (FAST 2011)*, volume 7140 of *Lecture Notes in Computer Science*, pages 182–197. Springer, Berlin, Heidelberg, 2012. doi:10.1007/978-3-642-29420-4_12.

9 Martin Hofmann and Wei Chen. Abstract interpretation from Büchi automata. In *Computer Science Logic and Logic in Computer Science (CSL-LICS 2014)*, pages 51:1–51:10. Association for Computing Machinery, 2014. doi:10.1145/2603088.2603127.

10 Martin Hofmann and Wei Chen. Büchi types for infinite traces and liveness, 2014. Technical report, arXiv:1401.5107 [cs.LO].

11 Gerard J Holzmann. *The SPIN Model Checker: Primer and Reference Manual*. Addison-Wesley Professional, Boston, MA, 2003.

12 Atsushi Igarashi, Benjamin C. Pierce, and Philip Wadler. Featherweight Java: a minimal core calculus for Java and GJ. *ACM Transactions on Programming Languages and Systems*, 23(3):396–450, 2001. doi:10.1145/503502.503505.

13 Microsoft. TypeScript - Typed JavaScript at any scale. URL: https://www.typescriptlang.org/.

14 Dominic Orchard and Nobuko Yoshida. Effects as sessions, sessions as effects. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL'16)*, page 568–581. Association for Computing Machinery, 2016. doi:10.1145/2837614.2837634.

15 McGill University Sable Group. Soot - A framework for analyzing and transforming Java and Android applications. URL: https://soot-oss.github.io/soot/.

16 Bolong Zeng and Li Tan. Test reactive systems with Büchi-automaton-based temporal requirements. In T. Bouabana-Tebibel and S. Rubin, editors, *Theoretical Information Reuse and Integration*, volume 446 of *Advances in Intelligent Systems and Computing*, pages 31–57. Springer, Cham, 2016. doi:10.1007/978-3-319-31311-5_2.

A Operational Semantics of Featherweight Java

In the operational semantics rules, we use an auxiliary function $\mathrm{classOf}_h(v)$ to determine the type of value v in heap h which is defined as follows:

$$\mathrm{classOf}_h(v) := \begin{cases} \texttt{NullType} & \text{if } v = \mathit{null} \\ C & \text{if } h(v) = (C, _) \in \mathit{Obj}. \end{cases}$$

The operational semantics of terminating evaluation and of exception throwing are defined mutually. They are generated inductively from the following rules:

$$\frac{}{(s,h) \vdash \texttt{emit}(a) \Downarrow \mathit{null}, h \,\&\, a} \qquad \frac{}{(s,h) \vdash x \Downarrow s(x), h \,\&\, \varepsilon} \qquad \frac{}{(s,h) \vdash \texttt{null} \Downarrow \mathit{null}, h \,\&\, \varepsilon}$$

$$\frac{s(x) = s(y) \qquad (s,h) \vdash e_1 \Downarrow v, h' \,\&\, w}{(s,h) \vdash \texttt{if } x = y \texttt{ then } e_1 \texttt{ else } e_2 \Downarrow v, h' \,\&\, w}$$

$$\frac{s(x) \neq s(y) \qquad (s,h) \vdash e_2 \Downarrow v, h' \,\&\, w}{(s,h) \vdash \texttt{if } x = y \texttt{ then } e_1 \texttt{ else } e_2 \Downarrow v, h' \,\&\, w}$$

$$\frac{(s,h) \vdash e_1 \Downarrow v_1, h_1 \,\&\, w_1 \qquad (s[x \mapsto v_1], h_1) \vdash e_2 \Downarrow v_2, h_2 \,\&\, w_2}{(s,h) \vdash \texttt{let } x = e_1 \texttt{ in } e_2 \Downarrow v_2, h_2 \,\&\, w_1 w_2}$$

$$\frac{l \notin \mathrm{dom}(h) \qquad G = [f \mapsto \mathit{null}]_{f \in \mathit{fields}(C)}}{(s,h) \vdash \texttt{new } C \Downarrow l, h[l \mapsto (C, G)] \,\&\, \varepsilon} \qquad \frac{(s,h) \vdash e \Downarrow v, h' \,\&\, w \qquad \mathrm{classOf}_{h'}(v) \preceq C}{(s,h) \vdash (C)\, e \Downarrow v, h' \,\&\, w}$$

$$\frac{s(x) = l \qquad h(l) = (_, G)}{(s,h) \vdash x.f \Downarrow G(f), h \,\&\, \varepsilon} \qquad \frac{s(x) = l \qquad h(l) = (D, G) \qquad h' = h[l \mapsto (D, G[f \mapsto s(y)])]}{(s,h) \vdash x.f := y \Downarrow s(y), h' \,\&\, \varepsilon}$$

$$\frac{s(x) = l \qquad h(l) = (D, _) \qquad \mathit{mtable}(D, m) = (\bar{x}, e) \qquad ([\texttt{this} \mapsto l] \cup [x_i \mapsto s(y_i)]_{i \in \{1,\ldots,|\bar{x}|\}}, h) \vdash e \Downarrow v, h' \,\&\, w}{(s,h) \vdash x.m(\bar{y}) \Downarrow v, h' \,\&\, w}$$

$$\frac{(s,h) \vdash e_1 \Downarrow v, h' \,\&\, w}{(s,h) \vdash \texttt{try } e_1 \texttt{ catch}(E\, x)\ e_2 \Downarrow v, h' \,\&\, w}$$

$$\frac{(s,h) \vdash e_1 \downharpoonright l, h_1 \,\&\, w_1 \qquad \mathrm{classOf}_{h_1}(l) \preceq E \qquad (s[x \mapsto l], h_1) \vdash e_2 \Downarrow v, h_2 \,\&\, w_2}{(s,h) \vdash \texttt{try } e_1 \texttt{ catch}(E\, x)\ e_2 \Downarrow v, h_2 \,\&\, w_1 w_2}$$

$$\frac{s(x) = s(y) \qquad (s,h) \vdash e_1 \downharpoonright l, h' \,\&\, w}{(s,h) \vdash \texttt{if } x = y \texttt{ then } e_1 \texttt{ else } e_2 \downharpoonright l, h' \,\&\, w}$$

$$\frac{s(x) \neq s(y) \qquad (s,h) \vdash e_2 \downharpoonright l, h' \,\&\, w}{(s,h) \vdash \texttt{if } x = y \texttt{ then } e_1 \texttt{ else } e_2 \downharpoonright l, h' \,\&\, w}$$

$$\frac{(s,h) \vdash e_1 \downharpoonright l, h' \,\&\, w}{(s,h) \vdash \texttt{let } x = e_1 \texttt{ in } e_2 \downharpoonright l, h' \,\&\, w} \qquad \frac{(s,h) \vdash e_1 \Downarrow v_1, h_1 \,\&\, w_1 \qquad (s[x \mapsto v_1], h_1) \vdash e_2 \downharpoonright l, h_2 \,\&\, w_2}{(s,h) \vdash \texttt{let } x = e_1 \texttt{ in } e_2 \downharpoonright l, h_2 \,\&\, w_1 w_2}$$

$$\frac{(s,h) \vdash e \downharpoonright l, h' \,\&\, w}{(s,h) \vdash (C)\, e \downharpoonright l, h' \,\&\, w} \qquad \frac{s(x) = l \qquad h(l) = (C, _) \qquad \mathit{mtable}(C, m) = (\bar{z}, e) \qquad ([\texttt{this} \mapsto l] \cup [z_i \mapsto s(y_i)]_{i \in \{1,\ldots,|\bar{z}|\}}, h) \vdash e \downharpoonright l', h' \,\&\, w}{(s,h) \vdash x.m(\bar{y}) \downharpoonright l', h' \,\&\, w}$$

$$\frac{(s,h) \vdash e \lfloor l, h' \,\&\, w}{(s,h) \vdash \texttt{throw}\ e \lfloor l, h' \,\&\, w} \qquad \frac{(s,h) \vdash e \Downarrow l, h' \,\&\, w}{(s,h) \vdash \texttt{throw}\ e \lfloor l, h' \,\&\, w}$$

$$\frac{(s,h) \vdash e_1 \lfloor l, h' \,\&\, w \qquad \mathrm{classOf}_{h'}(l) \not\preceq E}{(s,h) \vdash \texttt{try}\ e_1\ \texttt{catch}(E\,x)\ e_2 \lfloor l, h' \,\&\, w}$$

$$\frac{(s,h) \vdash e_1 \lfloor l_1, h_1 \,\&\, w_1 \qquad \mathrm{classOf}_{h_1}(l_1) \preceq E \qquad (s[x \mapsto l_1], h_1) \vdash e_2 \lfloor l_2, h_2 \,\&\, w_2}{(s,h) \vdash \texttt{try}\ e_1\ \texttt{catch}(E\,x)\ e_2 \lfloor l_2, h_2 \,\&\, w_1 w_2}$$

The one of nonterminating execution is generated coinductively from the following rules, where double horizontal lines are used:

$$\frac{\begin{array}{c} s(x) = s(y) \qquad (s,h) \vdash e_1 \Uparrow \,\&\, w \end{array}}{(s,h) \vdash \texttt{if}\ x = y\ \texttt{then}\ e_1\ \texttt{else}\ e_2 \Uparrow \,\&\, w} \qquad \frac{\begin{array}{c} s(x) \neq s(y) \qquad (s,h) \vdash e_2 \Uparrow \,\&\, w \end{array}}{(s,h) \vdash \texttt{if}\ x = y\ \texttt{then}\ e_1\ \texttt{else}\ e_2 \Uparrow \,\&\, w}$$

$$\frac{\begin{array}{c} (s,h) \vdash e_1 \Downarrow v_1, h_1 \,\&\, w_1 \\ (s[x \mapsto v_1], h_1) \vdash e_2 \Uparrow \,\&\, w_2 \end{array}}{(s,h) \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2 \Uparrow \,\&\, w_1 w_2} \qquad \frac{\begin{array}{c} (s,h) \vdash e_1 \Uparrow \,\&\, w \end{array}}{(s,h) \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2 \Uparrow \,\&\, w}$$

$$\frac{\begin{array}{c} (s,h) \vdash e \Uparrow \,\&\, w \end{array}}{(s,h) \vdash (C)\,e \Uparrow \,\&\, w} \qquad \frac{\begin{array}{c} s(x) = l \qquad h(l) = (C, _) \qquad \mathit{mtable}(C,m) = (\bar{x}, e) \\ ([\texttt{this} \mapsto l] \cup [x_i \mapsto s(y_i)]_{i \in \{1,\ldots,|\bar{x}|\}}, h) \vdash e \Uparrow \,\&\, w \end{array}}{(s,h) \vdash x.m(\bar{y}) \Uparrow \,\&\, w}$$

$$\frac{\begin{array}{c} (s,h) \vdash e \Uparrow \,\&\, w \end{array}}{(s,h) \vdash \texttt{throw}\ e \Uparrow \,\&\, w}$$

$$\frac{\begin{array}{c} (s,h) \vdash e_1 \Uparrow \,\&\, w \end{array}}{(s,h) \vdash \texttt{try}\ e_1\ \texttt{catch}(E\,x)\ e_2 \Uparrow \,\&\, w} \qquad \frac{\begin{array}{c} (s,h) \vdash e_1 \lfloor l, h_1 \,\&\, w_1 \qquad \mathrm{classOf}_{h_1}(l) \preceq E \\ (s[x \mapsto l], h_1) \vdash e_2 \Uparrow \,\&\, w_2 \end{array}}{(s,h) \vdash \texttt{try}\ e_1\ \texttt{catch}(E\,x)\ e_2 \Uparrow \,\&\, w_1 w_2}$$